

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



BÁO CÁO THỰC HÀNH
NHÓM NLP - K35
BÀI THỰC HÀNH SỐ 3

Thành viên nhóm

GVHD: ThS. Nguyễn Mạnh Hùng

1. Nguyễn Trọng Hiếu – 25C11042 (Nhóm trưởng)
2. Trương Lê Bảo Trân – 25C11071

TP. HỒ CHÍ MINH - 06/2026

Phân công công việc

Thành viên	Nhiệm vụ	Hoàn thành
25C11042 - Nguyễn Trọng Hiếu	- Nghiên cứu lý thuyết các mô hình YOLOv8, DETR và Faster R-CNN. - Thực hiện huấn luyện và đánh giá mô hình YOLOv8. - Thực hiện huấn luyện và đánh giá mô hình DETR. - Viết phần YOLOv8, DETR và chuẩn bị dữ liệu trong báo cáo.	100%
25C11071 - Trương Lê Bảo Trân	- Nghiên cứu lý thuyết các mô hình YOLOv8, DETR và Faster R-CNN. - Thực hiện huấn luyện và đánh giá mô hình Faster R-CNN. - Tổng hợp, phân tích và so sánh kết quả thực nghiệm của ba mô hình. - Viết phần Faster R-CNN, nhận xét, kết luận và hoàn thiện báo cáo..	100%

Bảng đánh giá nội dung đã hoàn thành

Nội dung	Chi tiết	Hoàn thành
Chọn tập dữ liệu	Thu thập thông tin tập dữ liệu	✓
	Mô tả tập dữ liệu	✓
	Cung cấp liên kết truy cập và tải dữ liệu	✓
YOLOv8	Cài đặt môi trường và thư viện	✓
	Huấn luyện mô hình	✓
	Đánh giá kết quả	✓
DETR	Cài đặt môi trường và thư viện	✓
	Huấn luyện mô hình	✓
	Đánh giá kết quả	✓
Faster R-CNN	Cài đặt môi trường và thư viện	✓
	Huấn luyện mô hình	✓
	Đánh giá kết quả	✓
So sánh mô hình	So sánh YOLOv8, DETR và Faster R-CNN	✓
	Phân tích ưu nhược điểm	✓
Báo cáo	Trình bày kết quả thực nghiệm	✓
	Nhận xét và kết luận	✓

Mục lục

Phân công công việc.....	i
Bảng đánh giá nội dung đã hoàn thành	ii
Mục lục.....	iii
Danh mục bảng biểu	vi
Chương 1. TỔNG QUAN BÀI THỰC HÀNH	1
1.1. Giới thiệu bài toán Object Detection.....	1
1.1.1. Khái niệm và ứng dụng của Object Detection	1
1.1.2. Quy trình cơ bản của bài toán Object Detection.....	2
1.1.3. Các tiêu chí đánh giá mô hình Object Detection	2
1.2. Mục tiêu bài thực hành	3
1.3. Giới thiệu tập dữ liệu Helmet	3
1.3.1. Nguồn dữ liệu	3
1.3.2. Thống kê dữ liệu	4
1.3.3. Các nhãn đối tượng	5
1.3.4. Định dạng dữ liệu	5
Chương 2. CƠ SỞ LÝ THUYẾT	7
2.1. YOLOv8.....	7
2.1.1. Kiến trúc YOLOv8	7
2.1.2. Nguyên lý hoạt động.....	7
2.2. DETR	8
2.2.1. Kiến trúc DETR	8
2.2.2. Nguyên lý hoạt động.....	9
2.3. Faster R-CNN.....	10
2.3.1. Kiến trúc Faster R-CNN	10
2.3.2. Nguyên lý hoạt động.....	10
Chương 3. THỰC NGHIỆM	12

3.1.	Môi trường thực nghiệm	12
3.2.	Chuẩn bị dữ liệu.....	12
3.2.1.	Dữ liệu cho YOLOv8.....	12
3.2.2.	Dữ liệu cho DETR và Faster R-CNN	13
3.3.	Huấn luyện mô hình YOLOv8.....	15
3.3.1.	Quy trình thực hiện	15
3.3.2.	Kết quả huấn luyện	18
3.4.	Huấn luyện mô hình DETR	22
3.4.1.	Quy trình thực hiện	22
3.4.2.	Kết quả huấn luyện	27
3.5.	Huấn luyện mô hình Faster R-CNN.....	30
3.5.1.	Quy trình thực hiện	30
3.5.2.	Kết quả đánh giá	34
3.6.	So sánh các mô hình YOLOv8, DETR và Faster R-CNN	36
3.6.1.	So sánh đặc điểm của các mô hình	36
3.6.2.	So sánh kết quả thực nghiệm trên bộ dữ liệu Safety Helmet.....	38
Chương 4.	KẾT LUẬN.....	40
	TÀI LIỆU THAM KHẢO	41

Danh mục hình ảnh

Hình 1. Minh họa một ví dụ về dữ liệu sau khi được gán nhãn.....	5
Hình 2. Cấu trúc thư mục của bộ dữ liệu Safety Helmet Dataset ở định dạng YOLOv8	12
Hình 3. Minh họa nội dung của một tệp nhãn trong định dạng YOLO. Mỗi dòng trong tệp tương ứng với một đối tượng xuất hiện trong ảnh và được biểu diễn bởi mã lớp đối tượng cùng các thông tin về vị trí Bounding Box đã được chuẩn hóa.	13
Hình 4. Cấu trúc thư mục của bộ dữ liệu Safety Helmet Dataset ở định dạng COCO.....	14
Hình 5. Minh họa cấu trúc dữ liệu trong tệp annotation theo chuẩn COCO	15
Hình 6. Learning Curve của mô hình YOLOv8.....	19
Hình 7. Confusion Matrix của mô hình YOLOv8	20
Hình 8. Learning Curve của mô hình DETR trên tập Validation.....	28
Hình 9. Sự thay đổi Training Loss trong quá trình huấn luyện Faster R-CNN	34

Danh mục bảng biểu

Bảng 1. Thống kê số lượng ảnh trong các tập Train, Validation và Test của bộ dữ liệu Helmet	4
Bảng 2. Kết quả tốt nhất của mô hình YOLOv8 trên tập Validation	21
Bảng 3. Kết quả của mô hình YOLOv8 trên tập Test	21
Bảng 4. Kết quả AP50-95 theo từng lớp đối tượng của mô hình DETR trên tập Validation	28
Bảng 5. Kết quả đánh giá mô hình DETR trên tập Validation.....	29
Bảng 6. Kết quả đánh giá mô hình DETR trên tập Test	29
Bảng 7. Kết quả đánh giá mô hình Faster R-CNN trên tập Validation.....	35
Bảng 8. Kết quả đánh giá mô hình Faster R-CNN trên tập Test.....	35
Bảng 9. Bảng so sánh tổng quan giữa các mô hình	37
Bảng 10. So sánh kết quả đánh giá trên tập Test	38

Chương 1. TỔNG QUAN BÀI THỰC HÀNH

1.1. Giới thiệu bài toán Object Detection

1.1.1. Khái niệm và ứng dụng của Object Detection

Object Detection (phát hiện đối tượng) là một trong những bài toán quan trọng của lĩnh vực Thị giác máy tính (Computer Vision), có nhiệm vụ xác định đồng thời vị trí và nhãn của các đối tượng xuất hiện trong ảnh hoặc video [1]. Khác với bài toán phân loại ảnh (Image Classification) chỉ dự đoán một nhãn cho toàn bộ ảnh, Object Detection yêu cầu mô hình vừa xác định lớp đối tượng vừa dự đoán tọa độ của các khung giới hạn (Bounding Box) tương ứng [1].

Trong những năm gần đây, sự phát triển mạnh mẽ của các phương pháp học sâu đã giúp nâng cao đáng kể hiệu quả của các hệ thống phát hiện đối tượng. Nhiều kiến trúc hiện đại đã được đề xuất nhằm cải thiện cả độ chính xác và tốc độ xử lý, cho phép triển khai trong các ứng dụng thực tế với quy mô lớn.

Theo các nghiên cứu tổng quan về Object Detection, các phương pháp phát hiện đối tượng truyền thống thường được chia thành hai nhóm chính gồm One-stage Detector và Two-stage Detector [1]. Bên cạnh đó, sự phát triển của kiến trúc Transformer đã dẫn đến sự ra đời của các mô hình mới như DETR (Detection Transformer), mở ra một hướng tiếp cận khác trong bài toán phát hiện đối tượng [3].

- **One-stage Detector:** Thực hiện đồng thời việc phát hiện và phân loại đối tượng trong một bước duy nhất. Nhóm phương pháp này có tốc độ xử lý nhanh, phù hợp với các ứng dụng thời gian thực. Đại diện tiêu biểu là họ mô hình YOLO (You Only Look Once) [1].

- **Two-stage Detector:** Thực hiện phát hiện đối tượng qua hai giai đoạn, bao gồm đề xuất vùng chứa đối tượng (Region Proposal) và phân loại đối tượng. Nhóm phương pháp này thường đạt độ chính xác cao hơn nhưng có tốc độ xử lý chậm hơn. Faster R-CNN là đại diện tiêu biểu của hướng tiếp cận này [1].

- **Transformer-based Detector:** Sử dụng kiến trúc Transformer và cơ chế Attention để thực hiện phát hiện đối tượng. Các mô hình thuộc nhóm này, điển hình là DETR, giúp đơn giản hóa quy trình phát hiện đối tượng bằng cách loại bỏ sự phụ thuộc vào các thành phần thủ công như Anchor Box và Non-Maximum Suppression (NMS) [3].

Object Detection được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau như:

- Hệ thống giám sát giao thông và phát hiện vi phạm.

- Xe tự hành và hệ thống hỗ trợ lái xe thông minh.
- Giám sát an ninh bằng camera.
- Nhận diện và theo dõi đối tượng trong video.
- Kiểm tra chất lượng sản phẩm trong công nghiệp.
- Phân tích hình ảnh y tế hỗ trợ chẩn đoán bệnh.

1.1.2. Quy trình cơ bản của bài toán Object Detection

Mặc dù các mô hình Object Detection có kiến trúc khác nhau, hầu hết đều tuân theo một quy trình xử lý chung gồm các bước như sau:

Input Image → *Feature Extraction* → *Object Detection Head* → *Classification & Bounding Box Regression* → *Detection Results (Bounding Boxes, Class Labels, Confidence Scores)*

Ở bước đầu tiên, mô hình nhận một ảnh đầu vào chứa một hoặc nhiều đối tượng cần phát hiện. Ảnh sau đó được đưa qua mạng CNN hoặc các kiến trúc hiện đại như Transformer để trích xuất đặc trưng (Feature Extraction). Các đặc trưng này biểu diễn thông tin về hình dạng, màu sắc, kết cấu và vị trí của các đối tượng xuất hiện trong ảnh.

Từ các đặc trưng đã được trích xuất, mô hình sử dụng **Object Detection Head** để đồng thời dự đoán vị trí và lớp của các đối tượng trong ảnh. Trong giai đoạn **Classification & Bounding Box Regression**, mô hình thực hiện hai nhiệm vụ chính:

- **Classification:** dự đoán nhãn lớp của đối tượng.
- **Bounding Box Regression:** dự đoán tọa độ của khung giới hạn (Bounding Box) bao quanh đối tượng.

Ngoài ra, mô hình còn dự đoán **Confidence Score**, thể hiện mức độ tin cậy của kết quả phát hiện.

Kết quả cuối cùng của bài toán Object Detection là tập hợp các **Bounding Boxes**, **Class Labels** và **Confidence Scores** tương ứng với các đối tượng được phát hiện trong ảnh. Ví dụ, đối với bài toán phát hiện mũ bảo hiểm, mô hình có thể xác định được vị trí và nhãn của các đối tượng **Person**, **Head** và **Helmet** xuất hiện trong ảnh.

1.1.3. Các tiêu chí đánh giá mô hình Object Detection

Để đánh giá hiệu quả của các mô hình phát hiện đối tượng, một số chỉ số thường được sử dụng bao gồm:

- **Precision:** Tỷ lệ các đối tượng được dự đoán đúng trên tổng số các đối tượng mà mô hình dự đoán.
- **Recall:** Tỷ lệ các đối tượng thực tế được mô hình phát hiện thành công.
- **mAP@0.5 (Mean Average Precision):** Giá trị trung bình của Average Precision khi ngưỡng IoU được đặt bằng 0.5.
- **mAP@0.5:0.95:** Giá trị trung bình của Average Precision trên nhiều ngưỡng IoU từ 0.5 đến 0.95 với bước nhảy 0.05, phản ánh toàn diện hơn chất lượng phát hiện đối tượng.

Trong đó, **IoU (Intersection over Union)** là chỉ số đo mức độ chồng lấp giữa Bounding Box dự đoán và Bounding Box thực tế. Giá trị IoU càng cao cho thấy vị trí đối tượng được dự đoán càng chính xác.

1.2. Mục tiêu bài thực hành

Bài thực hành được thực hiện nhằm các mục tiêu sau:

- Tìm hiểu bài toán phát hiện đối tượng (Object Detection) và các tiêu chí đánh giá thường được sử dụng.
- Nghiên cứu nguyên lý hoạt động của các mô hình YOLOv8, DETR và Faster R-CNN.
- Triển khai, huấn luyện và đánh giá các mô hình YOLOv8, DETR và Faster R-CNN trên tập dữ liệu Helmet Detection.
- Đánh giá hiệu năng của các mô hình. So sánh khả năng phát hiện đối tượng giữa các mô hình trên cùng một tập dữ liệu.
- Phân tích ưu điểm, hạn chế và đặc điểm của từng phương pháp trong bài toán phát hiện đối tượng.

1.3. Giới thiệu tập dữ liệu Helmet

1.3.1. Nguồn dữ liệu

Trong bài thực hành, nhóm sử dụng bộ dữ liệu **Safety Helmet Dataset** được công bố trên nền tảng Roboflow Universe. Đây là nền tảng chia sẻ dữ liệu mã nguồn mở phổ biến trong cộng đồng Computer Vision, cho phép người dùng truy cập, quản lý và xuất dữ liệu theo nhiều định dạng khác nhau phục vụ các tác vụ phát hiện đối tượng (Object Detection), phân loại ảnh (Image Classification) và phân đoạn ảnh (Image Segmentation).

Dataset được cung cấp tại:

Dataset gốc:

<https://universe.roboflow.com/dataperson/safety-helmet-dataset-uvh1t>

Link Drive lưu trữ phiên bản YOLOv8:

https://drive.google.com/file/d/1Ihd1CwR9k3XnLtF17f2w8fNbKMEZ7_k/view?usp=drive_link

Link Drive lưu trữ phiên bản COCO:

https://drive.google.com/file/d/1qHouId0FNqHbdBTdrj2kEpQHQT1wpEae/view?usp=drive_link

Bộ dữ liệu được xây dựng cho bài toán phát hiện các đối tượng liên quan đến việc đội mũ bảo hiểm trong môi trường giao thông. Mỗi ảnh được gán nhãn dưới dạng Bounding Box nhằm xác định vị trí và loại đối tượng xuất hiện trong ảnh.

1.3.2. Thống kê dữ liệu

Phiên bản dữ liệu được sử dụng trong bài thực hành là phiên bản V1 được phát hành trên Roboflow vào tháng 03 năm 2023. Theo thông tin do Roboflow cung cấp, dataset bao gồm tổng cộng 1088 ảnh và được chia sẵn thành ba tập dữ liệu như Bảng 1.

Bảng 1. Thống kê số lượng ảnh trong các tập Train, Validation và Test của bộ dữ liệu Helmet

Tập dữ liệu	Số lượng ảnh	Tỷ lệ
Train	761	70%
Validation	218	20%
Test	109	10%

Nhóm sử dụng trực tiếp cách phân chia dữ liệu do tác giả dataset cung cấp và không thực hiện tái phân chia dữ liệu. Trước khi xuất dữ liệu, tác giả dataset đã áp dụng các bước tiền xử lý sau:

Auto-Orient: Tự động điều chỉnh hướng ảnh dựa trên metadata EXIF nhằm đảm bảo tất cả ảnh có cùng chiều hiển thị.

Resize (Stretch to 416×416): Thay đổi kích thước toàn bộ ảnh về độ phân giải 416×416 pixel bằng phương pháp kéo giãn (stretch).

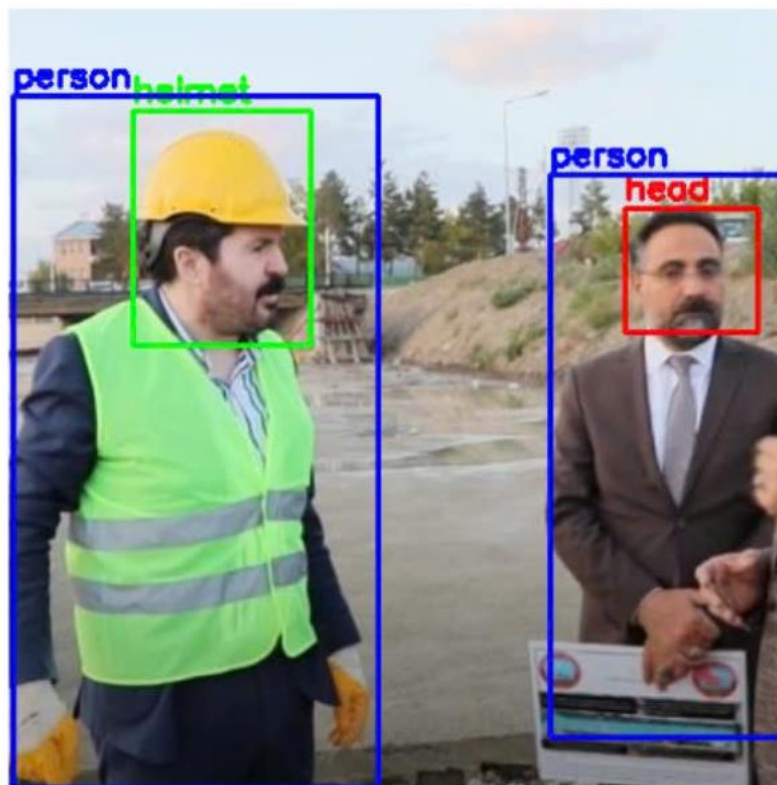
Việc chuẩn hóa kích thước ảnh về cùng một độ phân giải giúp đảm bảo tính nhất quán của dữ liệu đầu vào giữa các mô hình trong quá trình huấn luyện và đánh giá.

1.3.3. Các nhãn đối tượng

Dataset bao gồm ba lớp đối tượng:

- Helmet (Mũ bảo hiểm)
- Head (Đầu người)
- Person (Người)

Các đối tượng trong ảnh được gán nhãn dưới dạng Bounding Box, trong đó mỗi Bounding Box xác định vị trí của một đối tượng thông qua tọa độ của khung hình chữ nhật bao quanh đối tượng đó.



Hình 1. Minh họa một ví dụ về dữ liệu sau khi được gán nhãn.

Có thể quan sát rằng các đối tượng trong ảnh được bao quanh bởi các Bounding Box với màu sắc khác nhau, tương ứng với các lớp đối tượng đã được định nghĩa trong bộ dữ liệu. Các Bounding Box này cung cấp thông tin vị trí của đối tượng và được sử dụng làm dữ liệu nhãn trong quá trình huấn luyện mô hình.

1.3.4. Định dạng dữ liệu

Dataset được xuất ra dưới hai định dạng khác nhau nhằm phục vụ cho các mô hình được sử dụng trong bài thực hành.

Định dạng YOLO

Định dạng YOLO được sử dụng làm dữ liệu đầu vào cho mô hình YOLOv8. Mỗi ảnh đi kèm một tệp nhãn .txt chứa thông tin: ***class_id x_center y_center width height***. Trong đó các tọa độ được chuẩn hóa về khoảng giá trị $[0,1]$ theo kích thước ảnh.

Định dạng COCO

Định dạng COCO được sử dụng làm dữ liệu đầu vào cho các mô hình DETR và Faster R-CNN. Dữ liệu được lưu trong tệp JSON theo chuẩn COCO, bao gồm:

- Thông tin ảnh (images)
- Danh sách nhãn (categories)
- Thông tin annotation (annotations)
- Tọa độ Bounding Box
- ID ảnh và ID đối tượng

Trong quá trình thực nghiệm, nhóm không thực hiện các bước tiền xử lý thủ công như cân bằng dữ liệu, loại bỏ ảnh hoặc chỉnh sửa annotation. Nhóm sử dụng trực tiếp dữ liệu đã được tác giả dataset xử lý và xuất sẵn từ Roboflow.

Đối với YOLOv8, nhóm sử dụng phiên bản dữ liệu ở định dạng YOLOv8. Đối với DETR và Faster R-CNN, nhóm sử dụng phiên bản dữ liệu ở định dạng COCO nhằm phù hợp với yêu cầu đầu vào của từng mô hình.

Chương 2. CƠ SỞ LÝ THUYẾT

Trong bài thực hành, nhóm lựa chọn ba mô hình YOLOv8, Faster R-CNN và DETR nhằm đại diện cho các kiến trúc tiêu biểu trong bài toán Object Detection. Trong đó, YOLOv8 thuộc nhóm One-stage Detector, Faster R-CNN thuộc nhóm Two-stage Detector, còn DETR là mô hình phát hiện đối tượng dựa trên kiến trúc Transformer. Việc lựa chọn các mô hình này giúp đánh giá và so sánh hiệu quả của các hướng tiếp cận khác nhau trên cùng một tập dữ liệu.

2.1. YOLOv8

2.1.1. Kiến trúc YOLOv8

YOLOv8 là mô hình phát hiện đối tượng thuộc nhóm One-stage Detector, được phát triển bởi Ultralytics [2]. Khác với các phương pháp Two-stage Detector, YOLOv8 thực hiện đồng thời quá trình xác định vị trí và phân loại đối tượng trong một lần suy luận duy nhất, giúp đạt tốc độ xử lý cao và phù hợp với các ứng dụng thời gian thực [2].

Một trong những điểm nổi bật của YOLOv8 là việc sử dụng cơ chế Anchor-Free thay cho Anchor-Based như ở nhiều phiên bản YOLO trước đây. Cách tiếp cận này giúp giảm độ phức tạp trong quá trình huấn luyện, đồng thời cải thiện khả năng phát hiện đối tượng ở nhiều kích thước khác nhau.

Kiến trúc YOLOv8 gồm ba thành phần chính:

- **Backbone:** Có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. YOLOv8 sử dụng các lớp Convolution kết hợp với các khối C2f (Cross Stage Partial with Two Convolutions) nhằm tăng khả năng học đặc trưng và cải thiện hiệu quả lan truyền gradient trong quá trình huấn luyện.
- **Neck:** Có nhiệm vụ tổng hợp và kết hợp thông tin từ nhiều mức đặc trưng khác nhau. Thành phần này sử dụng cấu trúc Feature Pyramid Network (FPN) và Path Aggregation Network (PAN) giúp mô hình phát hiện hiệu quả các đối tượng có kích thước lớn, trung bình và nhỏ.
- **Detection Head:** Có nhiệm vụ dự đoán kết quả phát hiện đối tượng từ các đặc trưng đã được tổng hợp. Đầu ra của Detection Head bao gồm vị trí Bounding Box, nhãn lớp và độ tin cậy (Confidence Score) của từng đối tượng được phát hiện.

2.1.2. Nguyên lý hoạt động

Quy trình xử lý của YOLOv8 có thể được mô tả như sau:

Input Image → Backbone → Neck (FPN + PAN) → Detection Head → Detection Results (Bounding Boxes, Class Labels, Confidence Scores)

Đầu tiên, ảnh đầu vào được đưa qua Backbone để trích xuất các đặc trưng quan trọng. Các đặc trưng này chứa thông tin về hình dạng, màu sắc, kết cấu và vị trí của các đối tượng xuất hiện trong ảnh.

Tiếp theo, các đặc trưng được truyền đến Neck để tổng hợp thông tin từ nhiều mức độ khác nhau. Quá trình này giúp mô hình cải thiện khả năng phát hiện các đối tượng có kích thước khác nhau trong cùng một ảnh.

Sau đó, Detection Head sử dụng các đặc trưng đã được tổng hợp để đồng thời thực hiện ba nhiệm vụ:

- Dự đoán vị trí của đối tượng thông qua Bounding Box.
- Dự đoán nhãn lớp của đối tượng.
- Dự đoán độ tin cậy (Confidence Score) của kết quả phát hiện.

Khác với các mô hình Two-stage Detector, YOLOv8 thực hiện toàn bộ quá trình phát hiện và phân loại đối tượng trong một lần suy luận duy nhất. Điều này giúp giảm đáng kể thời gian xử lý và cho phép triển khai trong các ứng dụng yêu cầu tốc độ cao như giám sát giao thông, camera an ninh và hệ thống hỗ trợ lái xe thông minh.

2.2. DETR

2.2.1. Kiến trúc DETR

DETR (Detection Transformer) là mô hình phát hiện đối tượng được đề xuất bởi Carion và cộng sự tại hội nghị ECCV 2020 [3]. Khác với các phương pháp Object Detection truyền thống dựa trên Anchor Box và Non-Maximum Suppression (NMS), DETR tiếp cận bài toán phát hiện đối tượng dưới dạng một bài toán dự đoán tập hợp (Set Prediction), cho phép dự đoán trực tiếp các đối tượng xuất hiện trong ảnh theo cơ chế end-to-end [3].

Một trong những điểm nổi bật của DETR là việc đưa kiến trúc Transformer vào bài toán phát hiện đối tượng. Nhờ cơ chế Self-Attention, mô hình có khả năng học mối quan hệ giữa các vùng khác nhau trong ảnh, từ đó cải thiện khả năng nhận biết đối tượng và bối cảnh xung quanh.

Kiến trúc DETR gồm ba thành phần chính:

- **CNN Backbone:** Có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Trong bài thực hành này, nhóm sử dụng phiên bản DETR với Backbone ResNet-50 được

cung cấp bởi Hugging Face Transformers để tạo ra bản đồ đặc trưng (Feature Map) từ ảnh.

- **Transformer Encoder-Decoder:** Đây là thành phần cốt lõi của DETR. Transformer Encoder có nhiệm vụ học mối quan hệ giữa các vị trí khác nhau trên bản đồ đặc trưng thông qua cơ chế Self-Attention. Sau đó, Transformer Decoder sử dụng một tập các Object Queries để truy xuất thông tin và dự đoán các đối tượng tiềm năng trong ảnh.
- **Prediction Head:** Có nhiệm vụ dự đoán nhãn lớp và tọa độ Bounding Box của từng đối tượng được Transformer Decoder tạo ra.

Khác với các phương pháp Object Detection truyền thống, DETR không sử dụng Anchor Box, Region Proposal Network (RPN) hoặc Non-Maximum Suppression (NMS). Thay vào đó, mô hình sử dụng thuật toán Hungarian Matching trong quá trình huấn luyện để thiết lập tương ứng một-một giữa kết quả dự đoán và nhãn thực tế [3].

2.2.2. Nguyên lý hoạt động

Quy trình xử lý của DETR có thể được mô tả như sau:

Input Image → *CNN Backbone* → *Feature Map* → *Transformer Encoder* → *Transformer Decoder* → *Prediction Head* → *Detection Results (Bounding Boxes, Class Labels)*

Đầu tiên, ảnh đầu vào được đưa qua CNN Backbone để trích xuất đặc trưng. Kết quả thu được là một bản đồ đặc trưng (Feature Map) chứa thông tin về hình dạng, màu sắc và vị trí của các đối tượng trong ảnh.

Tiếp theo, Feature Map được đưa vào Transformer Encoder. Thông qua cơ chế Self-Attention, Encoder học được mối quan hệ giữa các vị trí khác nhau trên ảnh và xây dựng biểu diễn đặc trưng mang tính toàn cục (Global Representation).

Sau đó, Transformer Decoder nhận đầu vào từ Encoder cùng với một tập các Object Queries đã được khởi tạo sẵn. Mỗi Object Query có nhiệm vụ tìm kiếm một đối tượng tiềm năng trong ảnh. Kết quả của Decoder là tập các biểu diễn đặc trưng tương ứng với các đối tượng được phát hiện.

Cuối cùng, Prediction Head sử dụng các đặc trưng từ Decoder để dự đoán:

- Nhãn lớp của đối tượng.
- Tọa độ Bounding Box.
- Xác suất đối tượng tồn tại trong ảnh.

Trong quá trình huấn luyện, DETR sử dụng Hungarian Matching để ghép nối các dự đoán với nhãn thực tế. Cơ chế này giúp đảm bảo mỗi đối tượng chỉ được dự đoán một lần, từ đó loại bỏ nhu cầu sử dụng Non-Maximum Suppression như trong các phương pháp Object Detection truyền thống.

Nhờ khả năng mô hình hóa mối quan hệ toàn cục giữa các vùng trong ảnh và thiết kế end-to-end, DETR đạt hiệu quả cạnh tranh với các phương pháp phát hiện đối tượng truyền thống, đồng thời đơn giản hóa đáng kể quy trình phát hiện đối tượng. Tuy nhiên, DETR thường yêu cầu thời gian huấn luyện dài hơn và gặp khó khăn trong việc phát hiện các đối tượng có kích thước nhỏ.

2.3. Faster R-CNN

2.3.1. Kiến trúc Faster R-CNN

Faster R-CNN là mô hình phát hiện đối tượng thuộc nhóm Two-stage Detector, được đề xuất bởi Ren và cộng sự vào năm 2015 [4]. Khác với các mô hình One-stage Detector như YOLOv8, Faster R-CNN thực hiện phát hiện đối tượng thông qua hai giai đoạn riêng biệt gồm đề xuất vùng chứa đối tượng và phân loại đối tượng. Cách tiếp cận này giúp mô hình đạt độ chính xác cao trong nhiều bài toán phát hiện đối tượng.

Kiến trúc Faster R-CNN gồm bốn thành phần chính:

- **Backbone:** Có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Trong bài thực hành này, mô hình sử dụng mạng ResNet50 kết hợp với Feature Pyramid Network (FPN) để tạo ra các bản đồ đặc trưng ở nhiều mức độ khác nhau.
- **Region Proposal Network (RPN):** Có nhiệm vụ xác định các vùng trong ảnh có khả năng chứa đối tượng và tạo ra tập các vùng đề xuất (Region Proposals).
- **ROI Head:** Có nhiệm vụ trích xuất đặc trưng từ các vùng đề xuất và thực hiện phân loại đối tượng cũng như tính chỉnh tọa độ Bounding Box.
- **Fast R-CNN Head:** Có nhiệm vụ phân loại đối tượng và tính chỉnh tọa độ Bounding Box nhằm tạo ra kết quả phát hiện cuối cùng.

Trong bài thực hành này, nhóm sử dụng mô hình Faster R-CNN với Backbone ResNet50 kết hợp Feature Pyramid Network (FPN) được cung cấp bởi thư viện Torchvision. Mô hình được khởi tạo từ các trọng số đã được huấn luyện trước trên bộ dữ liệu COCO và thay thế lớp phân loại cuối cùng để phù hợp với ba lớp đối tượng gồm Helmet, Head và Person.

2.3.2. Nguyên lý hoạt động

Quy trình xử lý của Faster R-CNN có thể được mô tả như sau:

Input Image → *ResNet50 + FPN* → *Feature Maps* → *Region Proposal Network (RPN)* → *Region Proposals* → *ROI Pooling* → *Fast R-CNN Head* → *Detection Results (Bounding Boxes, Class Labels, Confidence Scores)*

Đầu tiên, ảnh đầu vào được đưa qua Backbone để trích xuất các đặc trưng quan trọng. Trong mô hình được sử dụng, Backbone bao gồm ResNet50 kết hợp với FPN, cho phép khai thác thông tin đặc trưng ở nhiều mức độ khác nhau nhằm hỗ trợ phát hiện các đối tượng có kích thước đa dạng.

Tiếp theo, các bản đồ đặc trưng được đưa vào Region Proposal Network (RPN). Thành phần này có nhiệm vụ xác định các vùng trong ảnh có khả năng chứa đối tượng và tạo ra tập các vùng đề xuất (Region Proposals).

Các vùng đề xuất sau đó được chuyển đến ROI Pooling để trích xuất đặc trưng và chuẩn hóa kích thước đầu vào. Các đặc trưng thu được tiếp tục được đưa vào Fast R-CNN Head để thực hiện hai nhiệm vụ:

- Phân loại đối tượng.
- Tính chỉnh tọa độ Bounding Box.

Cuối cùng, mô hình trả về kết quả phát hiện đối tượng bao gồm:

- Nhãn lớp của đối tượng.
- Tọa độ Bounding Box.
- Độ tin cậy (Confidence Score).

Nhờ việc tách riêng quá trình đề xuất vùng và phân loại đối tượng, Faster R-CNN có khả năng phát hiện đối tượng với độ chính xác cao trên nhiều bộ dữ liệu khác nhau. Tuy nhiên, do phải thực hiện hai giai đoạn xử lý liên tiếp nên tốc độ suy luận thường chậm hơn các mô hình One-stage Detector như YOLOv8.

Chương 3. THỰC NGHIỆM

3.1. Môi trường thực nghiệm

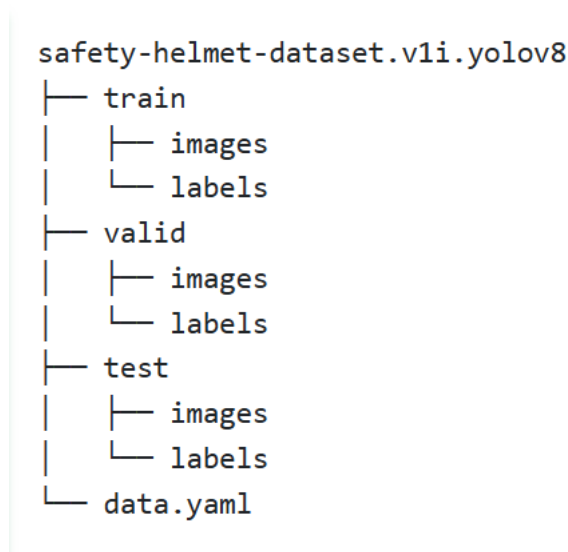
Các thí nghiệm trong báo cáo được thực hiện trên nền tảng Google Colab sử dụng ngôn ngữ lập trình Python 3.x. Các thư viện chính được sử dụng bao gồm Ultralytics YOLOv8, PyTorch, Torchvision, Transformers, NumPy, Pillow (PIL) và Matplotlib.

Ba mô hình YOLOv8, DETR và Faster R-CNN được huấn luyện và đánh giá trên cùng bộ dữ liệu Safety Helmet Dataset nhằm đảm bảo tính khách quan trong quá trình so sánh kết quả thực nghiệm.

3.2. Chuẩn bị dữ liệu

3.2.1. Dữ liệu cho YOLOv8

Đối với mô hình YOLOv8, nhóm sử dụng phiên bản dữ liệu được xuất theo định dạng YOLO từ Roboflow. Bộ dữ liệu được tổ chức theo cấu trúc thư mục chuẩn của YOLO như Hình 2.



Hình 2. Cấu trúc thư mục của bộ dữ liệu Safety Helmet Dataset ở định dạng YOLOv8

Trong đó:

- **train/images** chứa các ảnh dùng để huấn luyện mô hình.
- **train/labels** chứa các tệp nhãn tương ứng với ảnh huấn luyện.
- **valid/images** và **valid/labels** được sử dụng trong quá trình đánh giá mô hình trên tập Validation.
- **test/images** và **test/labels** được sử dụng để đánh giá kết quả cuối cùng trên tập Test.

- **data.yaml** chứa thông tin cấu hình dataset như đường dẫn dữ liệu, số lượng lớp và tên các lớp đối tượng.

Mỗi ảnh trong dataset được gán nhãn bằng một tệp văn bản (.txt) có cùng tên với ảnh tương ứng. Nội dung mỗi dòng trong tệp nhãn được lưu theo định dạng:

class_id x_center y_center width height

Trong đó:

- **class_id** là mã lớp đối tượng.
- **x_center, y_center** là tọa độ tâm của Bounding Box.
- **width, height** là chiều rộng và chiều cao của Bounding Box.

Tất cả các giá trị tọa độ đều được chuẩn hóa về khoảng giá trị từ 0 đến 1 theo kích thước ảnh. Đây là định dạng đầu vào tiêu chuẩn được mô hình YOLOv8 sử dụng trong quá trình huấn luyện và đánh giá.

Name	
helmet-2-  rf.f9d6b25aa9dc46ed0583cc8e7f3c02d3.txt	1 0.6466346153846154 0.40384615384615385 0.08653846153846154 0.11538461538461539
helmet-8-  rf.121938a55a8efd4ebc1dc77d752929f6.txt	2 0.6766826923076923 0.5336538461538461 0.1658653846153846 0.42788461538461536
helmet-10-  rf.eee985865a1a52407316f0c9ea7a9b3e.txt	
helmet-12-  rf.e060b7cfb30b033ca50fc1b9ed48efe7.txt	

Hình 3. Minh họa nội dung của một tệp nhãn trong định dạng YOLO. Mỗi dòng trong tệp tương ứng với một đối tượng xuất hiện trong ảnh và được biểu diễn bởi mã lớp đối tượng cùng các thông tin về vị trí Bounding Box đã được chuẩn hóa.

3.2.2. Dữ liệu cho DETR và Faster R-CNN

Đối với các mô hình DETR và Faster R-CNN, nhóm sử dụng phiên bản dữ liệu được xuất theo chuẩn COCO (Common Objects in Context). Cấu trúc thư mục của dataset được tổ chức như sau:

```

safety-helmet-dataset.v1i.coco
├── train
│   ├── _annotations.coco.json
│   └── images
├── valid
│   ├── _annotations.coco.json
│   └── images
└── test
    ├── _annotations.coco.json
    └── images

```

Hình 4. Cấu trúc thư mục của bộ dữ liệu Safety Helmet Dataset ở định dạng COCO

Trong đó:

- Thư mục **images** chứa các ảnh của từng tập dữ liệu.
- Tập **_annotations.coco.json** chứa toàn bộ thông tin nhãn theo chuẩn COCO.
- Tập **_annotations.coco.json** bao gồm các thành phần chính:

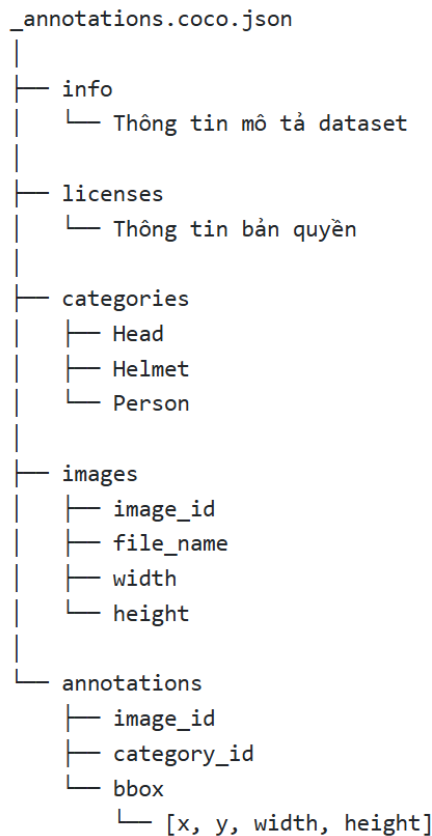
images: chứa thông tin của các ảnh trong dataset như tên ảnh, kích thước ảnh và ID ảnh.

annotations: chứa thông tin nhãn của các đối tượng xuất hiện trong ảnh, bao gồm ID ảnh, ID lớp đối tượng và tọa độ Bounding Box.

categories: chứa danh sách các lớp đối tượng trong dataset.

info và **licenses:** chứa các thông tin mô tả dataset do Roboflow cung cấp.

Trong đó, tọa độ Bounding Box được lưu trong trường **bbox** của mỗi annotation theo định dạng: **[x, y, width, height]** với (x, y) là tọa độ góc trên bên trái của Bounding Box, còn width và height lần lượt là chiều rộng và chiều cao của Bounding Box.



Hình 5. Minh họa cấu trúc dữ liệu trong tệp annotation theo chuẩn COCO

Khác với định dạng YOLO lưu nhãn trong các tệp văn bản (.txt) riêng lẻ, chuẩn COCO lưu toàn bộ thông tin ảnh và annotation trong một tệp JSON duy nhất. Định dạng này được sử dụng trực tiếp trong quá trình xây dựng Dataset và DataLoader cho các mô hình DETR và Faster R-CNN.

Trong quá trình thực nghiệm, nhóm giữ nguyên cách phân chia dữ liệu do Roboflow cung cấp và không thực hiện thêm các bước xử lý hoặc chỉnh sửa annotation thủ công. Điều này giúp đảm bảo tính nhất quán giữa các mô hình trong quá trình đánh giá và so sánh kết quả.

3.3. Huấn luyện mô hình YOLOv8

3.3.1. Quy trình thực hiện

Bước 1. Kiểm tra môi trường Colab

Nhóm tiến hành kiểm tra môi trường thực thi trên Google Colab nhằm xác nhận trạng thái GPU và bộ nhớ hệ thống. Kết quả cho thấy Colab sử dụng GPU NVIDIA Tesla T4 và CUDA được kích hoạt thành công, đáp ứng yêu cầu huấn luyện các mô hình Object Detection trong bài thực hành.

Bước 2. Kết nối Google Drive

Nhóm sử dụng Google Drive để lưu trữ bộ dữ liệu, mã nguồn và kết quả huấn luyện trong quá trình thực nghiệm. Thông qua thư viện google.colab, Google Drive được liên kết với môi trường Google Colab và được mount tại thư mục */content/gdrive*.

Bước 3. Cài đặt và khởi tạo YOLOv8

Nhóm tiến hành cài đặt thư viện **Ultralytics**, đây là thư viện chính thức cung cấp các mô hình YOLOv8 cùng các công cụ hỗ trợ huấn luyện, đánh giá và suy luận. Việc cài đặt được thực hiện thông qua lệnh *pip install ultralytics*.

Sau khi cài đặt thành công, lớp YOLO được import từ thư viện Ultralytics. Thư viện này cung cấp sẵn kiến trúc YOLOv8, các trọng số pretrained và các hàm phục vụ cho toàn bộ quy trình thực nghiệm như huấn luyện (train), đánh giá (val) và dự đoán (predict).

Sau bước này, môi trường thực nghiệm đã sẵn sàng để khởi tạo mô hình YOLOv8 và tiến hành huấn luyện trên bộ dữ liệu Safety Helmet Dataset.

Bước 4. Tải và chuẩn bị tập dữ liệu

Nhóm sử dụng bộ dữ liệu **Safety Helmet Dataset** đã được xuất sẵn từ nền tảng Roboflow theo định dạng YOLOv8. Ở bước này, chương trình kiểm tra sự tồn tại của dataset trong môi trường Colab. Nếu dữ liệu chưa tồn tại, bộ dữ liệu sẽ được tải từ Google Drive thông qua thư viện gdown và tự động giải nén vào thư mục làm việc.

Nhóm không thực hiện thêm các bước tiền xử lý dữ liệu thủ công trong quá trình thực nghiệm. Dataset được sử dụng trực tiếp từ phiên bản đã được Roboflow xử lý và xuất sẵn.

Sau bước này, toàn bộ dữ liệu huấn luyện, validation và test đã sẵn sàng để sử dụng trong quá trình huấn luyện mô hình YOLOv8.

Bước 5. Huấn luyện mô hình YOLOv8

Sau khi chuẩn bị dữ liệu, nhóm tiến hành huấn luyện mô hình YOLOv8 bằng hàm *train()* do thư viện Ultralytics cung cấp.

Mặc dù quá trình huấn luyện chỉ được thực hiện thông qua một lệnh duy nhất, bên trong thư viện Ultralytics đã tích hợp sẵn toàn bộ pipeline huấn luyện của YOLOv8, bao gồm:

- Đọc cấu hình dữ liệu từ tệp data.yaml.
- Tự động xây dựng Dataset và DataLoader.
- Tải mô hình YOLOv8 cùng trọng số pretrained.

- Thực hiện các kỹ thuật Data Augmentation trong quá trình huấn luyện.
- Tính toán hàm mất mát (Loss Function).
- Cập nhật trọng số bằng thuật toán tối ưu hóa.
- Đánh giá trên tập Validation sau mỗi epoch.
- Lưu checkpoint của mô hình.
- Tính toán các chỉ số Precision, Recall, mAP@0.5 và mAP@0.5:0.95.

Trong bài thực hành này, mô hình được khởi tạo từ tệp trọng số *yolov8s.pt* do Ultralytics cung cấp. Tệp này chứa kiến trúc YOLOv8s cùng các trọng số đã được huấn luyện trước trên bộ dữ liệu COCO với 80 lớp đối tượng. Nhờ đó, mô hình đã học được nhiều đặc trưng thị giác cơ bản và có thể được fine-tune trực tiếp trên bộ dữ liệu Safety Helmet Dataset thay vì phải huấn luyện từ đầu. Việc sử dụng trọng số pretrained giúp rút ngắn thời gian huấn luyện và cải thiện khả năng hội tụ của mô hình.

Trong quá trình huấn luyện, mô hình được đánh giá trên tập Validation sau mỗi epoch. Nếu chất lượng mô hình trên tập Validation không được cải thiện trong 20 epoch liên tiếp (*patience = 20*), cơ chế Early Stopping sẽ được kích hoạt nhằm tránh hiện tượng huấn luyện kéo dài không cần thiết và hạn chế nguy cơ overfitting.

Sau mỗi epoch, hệ thống tự động ghi nhận các giá trị Loss, Precision, Recall, mAP@0.5 và mAP@0.5:0.95. Trọng số của mô hình có kết quả tốt nhất trên tập Validation sẽ được lưu trong tệp best.pt để sử dụng cho các bước đánh giá và dự đoán sau này.

Bước 6. Trực quan hóa kết quả huấn luyện

Sau khi quá trình huấn luyện hoàn tất, nhóm tiến hành trực quan hóa các kết quả do YOLOv8 tự động sinh ra nhằm đánh giá khả năng hội tụ và chất lượng của mô hình.

- **Learning Curve:** Biểu đồ thể hiện sự thay đổi của các giá trị Loss và các chỉ số đánh giá trong suốt quá trình huấn luyện. Thông qua biểu đồ này có thể quan sát được xu hướng hội tụ của mô hình và đánh giá mức độ ổn định của quá trình học.
- **Confusion Matrix:** Ma trận nhầm lẫn được sử dụng để đánh giá khả năng phân biệt giữa các lớp đối tượng. Các giá trị trên đường chéo chính biểu thị số lượng dự đoán đúng của từng lớp, trong khi các giá trị ngoài đường chéo thể hiện các trường hợp dự đoán nhầm giữa các lớp.

Các biểu đồ trên được YOLOv8 tự động tạo ra và lưu trong thư mục kết quả huấn luyện, hỗ trợ quá trình phân tích và đánh giá mô hình trước khi thực hiện dự đoán trên tập dữ liệu kiểm thử.

Bước 7. Đánh giá mô hình trên tập Test

Sau khi quá trình huấn luyện hoàn tất, nhóm sử dụng tệp trọng số tốt nhất **best.pt** để đánh giá hiệu năng của mô hình trên tập dữ liệu Test. Việc đánh giá được thực hiện thông qua hàm **val()** của thư viện Ultralytics.

Trong bước này, mô hình tính toán các chỉ số đánh giá như Precision, Recall, mAP@0.5 và mAP@0.5:0.95 trên tập Test nhằm phản ánh khả năng phát hiện đối tượng của mô hình đối với dữ liệu chưa xuất hiện trong quá trình huấn luyện.

Bước 8. Dự đoán trên ảnh thử nghiệm

Sau khi đánh giá mô hình, nhóm tiến hành sử dụng mô hình đã huấn luyện để thực hiện dự đoán trên các ảnh trong tập Test thông qua hàm **predict()**.

Kết quả dự đoán bao gồm vị trí Bounding Box, nhãn lớp và độ tin cậy của từng đối tượng được phát hiện. Bước này giúp trực quan hóa khả năng nhận diện các đối tượng Helmet, Head và Person của mô hình trên dữ liệu thực tế.

Bước 9. Lưu mô hình đã huấn luyện

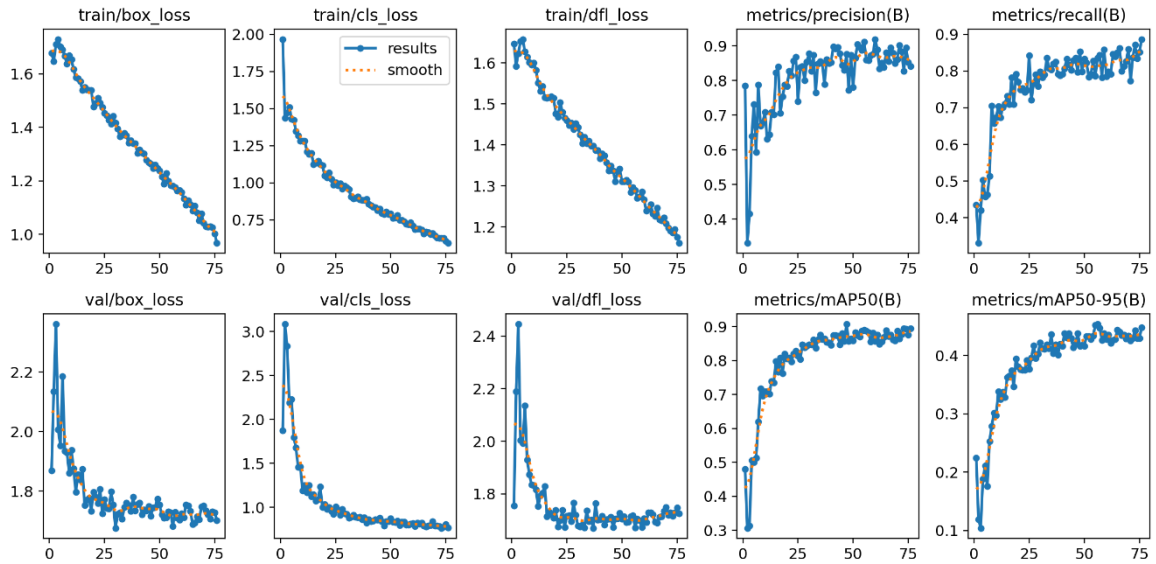
Sau khi hoàn thành quá trình huấn luyện và đánh giá, nhóm tiến hành lưu tệp trọng số tốt nhất (**best.pt**) vào Google Drive để phục vụ cho việc sử dụng lại mô hình trong các lần thực nghiệm tiếp theo.

3.3.2. Kết quả huấn luyện

Sau khi hoàn thành quá trình huấn luyện, YOLOv8 tự động ghi lại các giá trị Loss, Precision, Recall và mAP tại mỗi epoch để theo dõi quá trình học của mô hình. Thí nghiệm được cấu hình với tối đa 100 epoch, tuy nhiên cơ chế Early Stopping với tham số **patience** = 20 đã được kích hoạt khi kết quả trên tập Validation không còn cải thiện trong 20 epoch liên tiếp.

Kết quả huấn luyện cho thấy mô hình dừng tại epoch thứ 76, trong khi mô hình tốt nhất được ghi nhận tại epoch thứ 56 và tự động lưu vào tệp best.pt. Tổng thời gian huấn luyện khoảng 0,363 giờ ($\approx 21,8$ phút) trên GPU Tesla T4.

a) Learning Curve



Hình 6. Learning Curve của mô hình YOLOv8

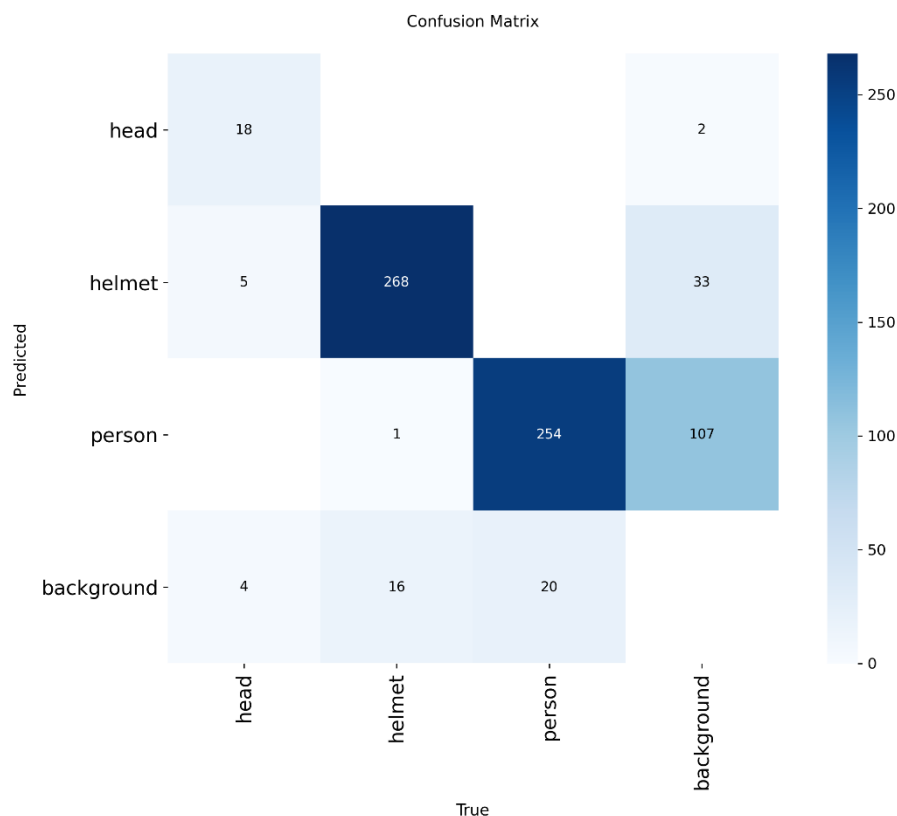
Quan sát Learning Curve cho thấy các giá trị **train/box_loss**, **train/cls_loss** và **train/dfl_loss** đều giảm liên tục trong suốt quá trình huấn luyện. Điều này cho thấy mô hình dần học được các đặc trưng của dữ liệu và quá trình tối ưu hóa diễn ra ổn định.

Đối với tập Validation, các giá trị **val/box_loss**, **val/cls_loss** và **val/dfl_loss** giảm mạnh trong giai đoạn đầu, đặc biệt trong khoảng 20 epoch đầu tiên, sau đó giảm chậm và dần hội tụ ở các epoch cuối. Khoảng cách giữa Train Loss và Validation Loss tương đối nhỏ, cho thấy mô hình có khả năng tổng quát hóa tốt và chưa xuất hiện dấu hiệu overfitting nghiêm trọng.

Các chỉ số đánh giá gồm **Precision**, **Recall**, **mAP@0.5** và **mAP@0.5:0.95** đều tăng nhanh trong giai đoạn đầu của quá trình huấn luyện. Sau khoảng **40–50 epoch**, các chỉ số bắt đầu ổn định và chỉ cải thiện rất ít ở các epoch tiếp theo. Kết thúc quá trình huấn luyện, mô hình đạt **Precision** xấp xỉ **0.89**, **Recall** khoảng **0.86**, **mAP@0.5** gần **0.89** và **mAP@0.5:0.95** khoảng **0.45** trên tập Validation.

Nhìn chung, các đường cong huấn luyện cho thấy mô hình YOLOv8 hội tụ tốt trên bộ dữ liệu Safety Helmet Dataset. Sự giảm dần của Loss cùng với sự gia tăng ổn định của các chỉ số đánh giá chứng tỏ mô hình học được các đặc trưng quan trọng của dữ liệu và đạt hiệu năng phát hiện đối tượng ổn định.

b) Confusion Matrix



Hình 7. Confusion Matrix của mô hình YOLOv8

Ma trận nhầm lẫn được sử dụng để đánh giá khả năng phân biệt giữa các lớp đối tượng trong tập dữ liệu. Có thể quan sát rằng phần lớn các giá trị tập trung trên đường chéo chính, cho thấy mô hình dự đoán đúng đa số các đối tượng thuộc ba lớp *Helmet*, *Head* và *Person*.

Trong đó, lớp **Helmet** đạt số lượng dự đoán đúng cao nhất với **268** đối tượng được nhận diện chính xác. Lớp **Person** cũng đạt kết quả tốt với **254** dự đoán đúng. Đối với lớp **Head**, mô hình dự đoán đúng **18** đối tượng. Mặc dù số lượng mẫu của lớp này thấp hơn đáng kể so với hai lớp còn lại, kết quả cho thấy mô hình vẫn có khả năng nhận diện tương đối tốt các đối tượng thuộc lớp Head.

Một số trường hợp nhầm lẫn vẫn xuất hiện giữa các lớp hoặc bị dự đoán thành **background**. Cụ thể, có **5** đối tượng Head bị dự đoán thành Helmet và **1** đối tượng Helmet bị dự đoán thành Person. Ngoài ra, một số đối tượng không được phát hiện bởi mô hình và được ghi nhận ở hàng background, bao gồm **4** đối tượng Head, **16** đối tượng Helmet và **20** đối tượng Person. Nguyên nhân chủ yếu đến từ các trường hợp đối tượng có kích thước nhỏ, bị che khuất hoặc xuất hiện trong điều kiện quan sát không thuận lợi.

Nhìn chung, ma trận nhầm lẫn cho thấy mô hình YOLOv8 có khả năng phân biệt tốt giữa các lớp đối tượng trong bộ dữ liệu Safety Helmet Dataset, đặc biệt đối với hai lớp

Helmet và Person. Điều này phù hợp với các chỉ số Precision, Recall và mAP đã đạt được trong quá trình đánh giá mô hình.

c) Kết quả đánh giá trên tập Validation

Bảng 2. Kết quả tốt nhất của mô hình YOLOv8 trên tập Validation

Chỉ số	Giá trị
Precision	0.859
Recall	0.835
mAP@0.5	0.889
mAP@0.5:0.95	0.454

Kết quả thực nghiệm cho thấy mô hình YOLOv8 đạt hiệu năng tốt trên tập Validation của bộ dữ liệu Safety Helmet Dataset. Giá trị **mAP@0.5** đạt **88.9%**, cho thấy phần lớn các đối tượng trong ảnh được phát hiện và định vị chính xác. Đồng thời, **Precision** đạt **85.9%** và **Recall** đạt **83.5%**, phản ánh khả năng cân bằng tương đối tốt giữa việc phát hiện đầy đủ đối tượng và hạn chế các dự đoán sai.

Giá trị **mAP@0.5:0.95** đạt **45.4%**, thấp hơn so với mAP@0.5 do tiêu chí đánh giá ở nhiều ngưỡng IoU nghiêm ngặt hơn. Tuy nhiên, kết quả này vẫn cho thấy mô hình có khả năng định vị đối tượng tương đối ổn định trên bộ dữ liệu thực nghiệm. Nhìn chung, YOLOv8 đã học được hiệu quả các đặc trưng của ba lớp đối tượng gồm **Head**, **Helmet** và **Person**, tạo cơ sở cho việc đánh giá trên tập Test và so sánh với các mô hình DETR và Faster R-CNN ở các phần tiếp theo.

d) Kết quả đánh giá mô hình tốt nhất trên tập Test

Sau khi lựa chọn mô hình tốt nhất (*best.pt*), nhóm tiến hành đánh giá trên tập Test nhằm kiểm tra khả năng tổng quát hóa của mô hình trên các dữ liệu chưa xuất hiện trong quá trình huấn luyện.

Bảng 3. Kết quả của mô hình YOLOv8 trên tập Test

Chỉ số	Giá trị
Precision	0.836

Recall	0.846
mAP@0.5	0.887
mAP@0.5:0.95	0.467

Kết quả cho thấy mô hình YOLOv8 đạt **mAP@0.5 = 88.7%** trên tập Test, đồng thời đạt **Precision = 83.6%** và **Recall = 84.6%**. Điều này cho thấy mô hình có khả năng phát hiện và định vị hiệu quả các đối tượng *Helmet*, *Head* và *Person* trên các ảnh chưa được sử dụng trong quá trình huấn luyện.

Bên cạnh đó, mô hình đạt **mAP@0.5:0.95 = 46.7%**, cho thấy chất lượng định vị Bounding Box vẫn được duy trì tương đối ổn định khi đánh giá ở nhiều ngưỡng IoU khác nhau. Kết quả này phản ánh khả năng tổng quát hóa tốt của mô hình trên bộ dữ liệu Safety Helmet Dataset.

Nhìn chung, YOLOv8 cho thấy khả năng hội tụ nhanh, hiệu năng ổn định và đạt độ chính xác cao trên bộ dữ liệu Safety Helmet Dataset. Kết quả thu được sẽ được sử dụng làm cơ sở để so sánh với các mô hình DETR và Faster R-CNN trong các phần thực nghiệm tiếp theo của báo cáo.

3.4. Huấn luyện mô hình DETR

3.4.1. Quy trình thực hiện

Bước 1. Kiểm tra môi trường

Nhóm tiến hành kiểm tra môi trường thực thi trên Google Colab nhằm xác nhận khả năng sử dụng GPU trong quá trình huấn luyện mô hình DETR. Chương trình sử dụng thư viện PyTorch để kiểm tra trạng thái CUDA và xác định thiết bị tính toán hiện tại.

Kết quả cho thấy môi trường Colab đã kích hoạt thành công GPU thông qua CUDA (device = cuda). Điều này cho phép tận dụng khả năng tính toán song song của GPU để tăng tốc quá trình huấn luyện mô hình DETR, đặc biệt đối với các phép tính liên quan đến Transformer và xử lý ảnh.

Sau bước này, môi trường thực nghiệm được xác nhận đủ điều kiện để tiến hành huấn luyện mô hình.

Bước 2. Kết nối Google Drive

Nhóm sử dụng Google Drive để lưu trữ bộ dữ liệu, mã nguồn và các trọng số mô hình trong quá trình thực nghiệm. Thông qua thư viện google.colab, Google Drive được kết nối với môi trường Colab và được mount tại thư mục */content/gdrive*.

Sau bước này, dữ liệu và các tệp kết quả có thể được đọc và ghi trực tiếp từ Google Drive trong suốt quá trình thực nghiệm.

Bước 3. Cài đặt mô hình DETR

Nhóm tiến hành cài đặt các thư viện cần thiết gồm transformers, datasets, timm và pycocotools để phục vụ cho việc xây dựng và huấn luyện mô hình DETR.

Tiếp theo, nhóm khởi tạo DetrImageProcessor từ mô hình *facebook/detr-resnet-50*. Thành phần này có nhiệm vụ tiền xử lý ảnh và annotation theo đúng định dạng đầu vào mà DETR yêu cầu.

Sau đó, nhóm xây dựng bảng ánh xạ giữa ID lớp và tên lớp tương ứng của bộ dữ liệu Safety Helmet Dataset gồm ba lớp: Head, Helmet và Person.

Mô hình DETR được khởi tạo từ phiên bản pretrained facebook/detr-resnet-50 do Hugging Face cung cấp. Mô hình này đã được huấn luyện trước trên bộ dữ liệu COCO và bao gồm Backbone ResNet-50, Transformer Encoder-Decoder cùng Detection Head phục vụ bài toán phát hiện đối tượng.

Do bộ dữ liệu Safety Helmet Dataset chỉ bao gồm ba lớp đối tượng, lớp phân loại cuối cùng của mô hình được thay thế để phù hợp với số lượng lớp của bài toán, trong khi các trọng số pretrained của Backbone và Transformer vẫn được giữ lại làm điểm khởi tạo ban đầu cho quá trình huấn luyện.

Cuối cùng, mô hình được chuyển sang GPU để phục vụ cho quá trình huấn luyện. Sau bước này, nhóm đã thu được mô hình DETR hoàn chỉnh với trọng số pretrained từ COCO, sẵn sàng cho các bước xây dựng Dataset và huấn luyện tiếp theo.

Bước 4. Tải và chuẩn bị tập dữ liệu

Nhóm sử dụng bộ dữ liệu Safety Helmet Dataset ở định dạng COCO. Dataset được tải từ Google Drive và giải nén vào môi trường Google Colab để phục vụ quá trình huấn luyện mô hình DETR.

Khác với YOLOv8 có thể đọc trực tiếp dữ liệu thông qua tệp *data.yaml*, DETR yêu cầu dữ liệu phải được chuyển đổi về đúng định dạng đầu vào của mô hình. Vì vậy, nhóm xây dựng lớp DetrCocoDataset kế thừa từ CocoDetection của Torchvision để đọc ảnh và annotation từ bộ dữ liệu COCO.

Trong lớp `DetrCocoDataset`, mỗi ảnh cùng các annotation tương ứng được xử lý bởi `DetrImageProcessor`. Thành phần này có nhiệm vụ:

- Đọc ảnh đầu vào.
- Chuyển đổi annotation COCO sang định dạng mà DETR yêu cầu.
- Chuẩn hóa dữ liệu ảnh.
- Tạo các tensor đầu vào phục vụ quá trình huấn luyện.

Kết quả đầu ra của mỗi mẫu dữ liệu bao gồm:

- `pixel_values`: ảnh đã được tiền xử lý và chuyển thành tensor.
- `labels`: thông tin annotation của ảnh dưới định dạng mà DETR sử dụng trong quá trình huấn luyện.

Do bộ dữ liệu Safety Helmet Dataset ở định dạng COCO có chứa một lớp trung gian ***person-head-helmet*** với `category_id = 0` không thuộc ba lớp đối tượng cần phát hiện, nhóm tiến hành loại bỏ lớp này trong quá trình xây dựng Dataset. Chỉ ba lớp Head, Helmet và Person được giữ lại để phục vụ quá trình huấn luyện và đánh giá mô hình.

Sau đó, nhóm tạo ba tập dữ liệu riêng biệt: `train_dataset`, `val_dataset`, `test_dataset`.

Để phục vụ quá trình huấn luyện theo mini-batch, nhóm xây dựng hàm ***collate_fn***. Hàm này có nhiệm vụ gộp nhiều mẫu dữ liệu thành một batch, đồng thời tạo thêm `pixel_mask` để DETR xử lý dữ liệu đầu vào một cách chính xác.

Sau bước này, toàn bộ dữ liệu đã được chuyển đổi sang định dạng phù hợp với mô hình DETR và sẵn sàng cho quá trình huấn luyện.

Bước 5. Huấn luyện mô hình DETR

Sau khi hoàn thành việc xây dựng Dataset và DataLoader, nhóm tiến hành huấn luyện mô hình DETR bằng lớp Trainer do thư viện Hugging Face Transformers cung cấp.

Trong quá trình cấu hình huấn luyện, mô hình được thiết lập với tối đa **100 epoch**, ***batch size bằng 2*** cho mỗi lần cập nhật dữ liệu. Đồng thời, nhóm sử dụng ***gradient_accumulation_steps = 8*** để tích lũy gradient qua nhiều mini-batch, tương đương ***batch size hiệu dụng khoảng 16 mẫu*** cho mỗi lần cập nhật trọng số.

Ngoài ra, chế độ tính toán dấu phẩy động 16-bit (fp16) được kích hoạt nhằm giảm mức sử dụng bộ nhớ và tăng tốc quá trình huấn luyện trên GPU.

Khác với YOLOv8 sử dụng hàm ***model.train()*** đã đóng gói toàn bộ pipeline huấn luyện trong thư viện Ultralytics, DETR yêu cầu người dùng phải tự xây dựng Dataset, DataLoader, hàm `collate_fn` và cấu hình quá trình huấn luyện trước khi truyền vào Trainer.

Mặc dù lệnh huấn luyện chỉ bao gồm: ***trainer.train()*** nhưng bên trong thư viện Transformers đã tích hợp toàn bộ quy trình huấn luyện, bao gồm:

- Đọc dữ liệu từ DataLoader.
- Thực hiện lan truyền xuôi (Forward Pass).
- Tính toán hàm mất mát (Loss Function).
- Thực hiện lan truyền ngược (Backpropagation).
- Cập nhật trọng số mô hình.
- Đánh giá trên tập Validation.
- Lưu checkpoint của mô hình.
- Theo dõi và ghi nhận các chỉ số huấn luyện.

Trong bài thực hành này, mô hình được đánh giá trên tập Validation sau mỗi epoch thông qua tham số ***eval_strategy="epoch"***. Đồng thời, checkpoint cũng được lưu sau mỗi epoch bằng tham số ***save_strategy="epoch"***.

Để lựa chọn mô hình tốt nhất, nhóm sử dụng giá trị `eval_loss` trên tập Validation làm tiêu chí đánh giá (`metric_for_best_model="eval_loss"`). Khi quá trình huấn luyện kết thúc, hệ thống tự động tải lại checkpoint có giá trị Validation Loss thấp nhất thông qua tham số ***load_best_model_at_end=True***.

Bên cạnh đó, cơ chế Early Stopping được áp dụng với ***early_stopping_patience = 20***. Nếu giá trị Validation Loss không được cải thiện trong 20 epoch liên tiếp, quá trình huấn luyện sẽ tự động dừng nhằm tránh lãng phí thời gian tính toán và hạn chế hiện tượng overfitting.

Sau khi hoàn tất huấn luyện, đường dẫn của checkpoint tốt nhất được lưu lại để sử dụng cho các bước đánh giá và dự đoán ở các phần tiếp theo.

Bước 6: Đánh giá kết quả huấn luyện

- Đánh giá mô hình trên tập Validation

Sau khi quá trình huấn luyện hoàn tất, nhóm tiến hành đánh giá mô hình DETR trên tập Validation nhằm theo dõi chất lượng mô hình và tạo cơ sở so sánh với YOLOv8.

Để đánh giá mô hình, nhóm sử dụng các chỉ số `mAR@100`, `mAP@0.5` và `mAP@0.5:0.95` được tính toán thông qua lớp MeanAveragePrecision của thư viện

TorchMetrics. Trong đó, $mAR@100$ phản ánh khả năng thu hồi đối tượng của mô hình khi xem xét tối đa 100 dự đoán trên mỗi ảnh.

Trong quá trình đánh giá, dữ liệu từ tập Validation được đưa qua mô hình để tạo ra các dự đoán. Sau đó, các kết quả dự đoán được chuyển đổi sang định dạng Bounding Box tiêu chuẩn và so sánh với nhãn thực tế của dataset. Từ các cặp dự đoán và nhãn thực tế, hệ thống tính toán các chỉ số đánh giá cuối cùng.

Kết quả thu được phản ánh khả năng phát hiện đối tượng của mô hình DETR trên dữ liệu chưa tham gia trực tiếp vào quá trình huấn luyện.

- **Learning Curve**

Để đánh giá khả năng hội tụ của mô hình, nhóm tiến hành trực quan hóa lịch sử huấn luyện được lưu trong `trainer.state.log_history`.

Trong quá trình huấn luyện, Hugging Face Trainer tự động ghi nhận giá trị Train Loss và Validation Loss sau mỗi epoch. Các giá trị này được trích xuất và sử dụng để xây dựng biểu đồ Learning Curve.

Biểu đồ Learning Curve cho phép quan sát sự thay đổi của Loss theo thời gian huấn luyện, từ đó đánh giá mức độ hội tụ của mô hình và phát hiện các dấu hiệu overfitting hoặc underfitting nếu có.

Kết quả này đóng vai trò tương tự biểu đồ Loss Curve được sử dụng trong quá trình đánh giá YOLOv8.

- **Đánh giá hiệu năng theo từng lớp đối tượng**

Ngoài các chỉ số đánh giá tổng thể như $mAR@100$, $mAP@0.5$ và $mAP@0.5:0.95$, nhóm tiếp tục đánh giá hiệu năng của mô hình DETR trên từng lớp đối tượng riêng biệt. Trong bước này, chỉ số AP50-95 được tính cho các lớp Head, Helmet và Person bằng thư viện TorchMetrics. Sau khi mô hình sinh ra các bounding box dự đoán, kết quả được chuyển đổi về tọa độ thực tế của ảnh và so sánh với nhãn gốc để tính toán AP của từng lớp. Việc đánh giá theo từng lớp giúp xác định khả năng phát hiện của mô hình đối với từng loại đối tượng, đồng thời phân tích mức độ ảnh hưởng của từng lớp đến giá trị mAP tổng thể.

Bước 7. Đánh giá mô hình tốt nhất trên tập Test

Sau khi quá trình huấn luyện hoàn tất, nhóm sử dụng checkpoint tốt nhất được lựa chọn trong quá trình huấn luyện để đánh giá trên tập Test.

Tương tự như bước đánh giá trên tập Validation, nhóm sử dụng thư viện TorchMetrics và lớp MeanAveragePrecision để tính toán các chỉ số đánh giá của mô hình. Các ảnh trong

tập Test được đưa qua mô hình DETR để tạo ra các dự đoán, sau đó được so sánh với nhãn thực tế của dataset nhằm tính toán các chỉ số đánh giá.

Để đảm bảo tính nhất quán với quá trình đánh giá trên tập Validation, nhóm sử dụng các chỉ số $mAR@100$, $mAP@0.5$ và $mAP@0.5:0.95$ trên tập Test.

Kết quả trên tập Test phản ánh khả năng tổng quát hóa của mô hình đối với dữ liệu chưa từng xuất hiện trong quá trình huấn luyện và được sử dụng làm cơ sở chính để so sánh hiệu năng giữa các mô hình trong phần thực nghiệm.

Bước 8. Dự đoán trên ảnh thử nghiệm

Sau khi đánh giá mô hình trên tập Test, nhóm thực hiện dự đoán trên các ảnh thử nghiệm nhằm quan sát trực quan khả năng phát hiện đối tượng của mô hình DETR.

Trong bước này, một ảnh được chọn ngẫu nhiên từ tập Test và đưa vào mô hình DETR đã huấn luyện. Mô hình thực hiện dự đoán vị trí Bounding Box, nhãn lớp và độ tin cậy của từng đối tượng xuất hiện trong ảnh. Kết quả dự đoán được hiển thị trực tiếp nhằm hỗ trợ việc quan sát và phân tích khả năng phát hiện đối tượng của mô hình.

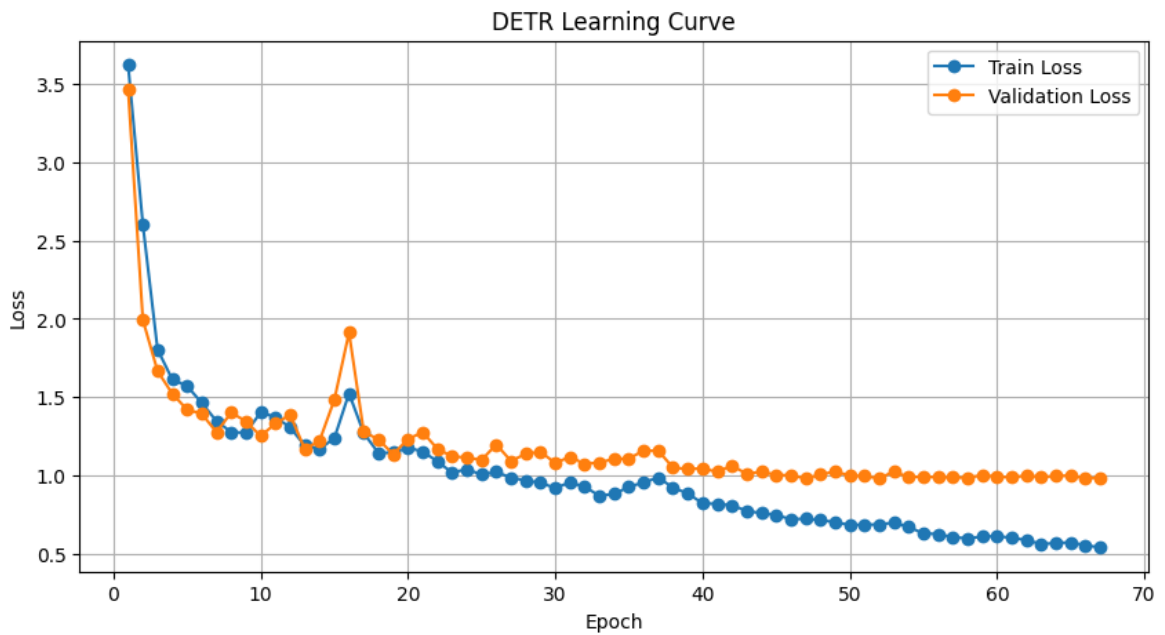
Bước 9. Lưu mô hình tốt nhất vào Google Drive

Sau khi quá trình huấn luyện hoàn tất, nhóm tiến hành lưu checkpoint tốt nhất của mô hình DETR vào Google Drive để phục vụ cho việc sử dụng và đánh giá sau này.

3.4.2. Kết quả huấn luyện

Sau khi hoàn thành quá trình huấn luyện, mô hình DETR được đánh giá trên tập Validation nhằm theo dõi khả năng học và mức độ tổng quát hóa của mô hình.

a) Learning Curve



Hình 8. Learning Curve của mô hình DETR trên tập Validation

Hình 8 cho thấy cả Train Loss và Validation Loss đều giảm mạnh trong những epoch đầu tiên, chứng tỏ mô hình đã nhanh chóng học được các đặc trưng quan trọng của bộ dữ liệu. Sau khoảng 20 epoch, tốc độ giảm của Loss chậm lại và các đường cong dần ổn định, cho thấy mô hình đang tiến tới trạng thái hội tụ.

Từ khoảng epoch 40 trở đi, Train Loss tiếp tục giảm trong khi Validation Loss gần như không còn cải thiện đáng kể và dao động quanh mức 1.0. Điều này cho thấy mô hình bắt đầu xuất hiện dấu hiệu overfitting nhẹ. Tuy nhiên, khoảng cách giữa hai đường Loss không quá lớn nên khả năng tổng quát hóa của mô hình vẫn được duy trì ở mức chấp nhận được. Việc huấn luyện được dừng tại epoch 67 nhờ cơ chế Early Stopping giúp hạn chế hiện tượng học quá mức trên tập huấn luyện.

b) Đánh giá hiệu năng theo từng lớp đối tượng

Bảng 4. Kết quả AP50-95 theo từng lớp đối tượng của mô hình DETR trên tập Validation

Lớp đối tượng	AP50-95
Head	0.000
Helmet	0.402
Person	0.432
mAP50-95	0.278

Kết quả đánh giá theo từng lớp cho thấy mô hình DETR đạt hiệu năng tương đối tốt đối với hai lớp Helmet và Person với AP50-95 lần lượt đạt 0.402 và 0.432. Ngược lại, lớp Head có AP50-95 bằng 0.000, cho thấy mô hình gần như không phát hiện được đối tượng thuộc lớp này. Điều này phản ánh hạn chế của DETR gốc trong việc xử lý các đối tượng có kích thước nhỏ trên bộ dữ liệu Safety Helmet. Hiệu năng thấp của lớp Head đã làm giảm đáng kể giá trị mAP50-95 tổng thể của mô hình xuống còn 0.278.

c) Kết quả đánh giá trên tập Validation

Bảng 5. Kết quả đánh giá mô hình DETR trên tập Validation

<i>Chỉ số</i>	<i>Giá trị</i>
<i>mAR@100</i>	<i>0.3550</i>
<i>mAP@0.5</i>	<i>0.5613</i>
<i>mAP@0.5:0.95</i>	<i>0.2780</i>

Kết quả đánh giá trên tập Validation cho thấy mô hình DETR đạt $mAR@100 = 0.3550$, $mAP@0.5 = 0.5613$ và $mAP@0.5:0.95 = 0.2780$. Giá trị $mAP@0.5$ ở mức khá cho thấy mô hình có khả năng phát hiện được các đối tượng trong ảnh. Tuy nhiên, $mAP@0.5:0.95$ còn tương đối thấp, cho thấy độ chính xác của các bounding box giảm khi yêu cầu ngưỡng IoU trở nên khắt khe hơn. Điều này phản ánh rằng mô hình vẫn gặp khó khăn trong việc định vị chính xác đối tượng, đặc biệt đối với các vật thể có kích thước nhỏ trong bộ dữ liệu Safety Helmet.

d) Kết quả đánh giá mô hình tốt nhất trên tập Test

Bảng 6. Kết quả đánh giá mô hình DETR trên tập Test

Chỉ số	Giá trị
mAR@100	0.3567
mAP@0.5	0.5317
mAP@0.5:0.95	0.2681

Trên tập Test, mô hình DETR đạt $mAR@100 = 0.3567$, $mAP@0.5 = 0.5317$ và $mAP@0.5:0.95 = 0.2681$. Các chỉ số này chỉ giảm nhẹ so với kết quả trên tập Validation ($mAP@0.5 = 0.5613$ và $mAP@0.5:0.95 = 0.2780$), cho thấy mô hình duy trì được khả năng tổng quát hóa tương đối ổn định trên dữ liệu chưa được sử dụng trong quá trình huấn luyện. Tuy nhiên, giá trị $mAP@0.5:0.95$ vẫn còn ở mức thấp, cho thấy mô hình gặp khó khăn trong việc định vị chính xác các đối tượng ở các ngưỡng IoU cao, đặc biệt đối với các vật thể có kích thước nhỏ trong bộ dữ liệu Safety Helmet.

3.5. Huấn luyện mô hình Faster R-CNN

3.5.1. Quy trình thực hiện

Bước 1. Chuẩn bị môi trường

Nhóm tiến hành cài đặt và kiểm tra môi trường thực thi trên Google Colab. Trong bước này, hệ thống tự động nhận diện thiết bị tính toán đang sử dụng (CPU hoặc GPU), đồng thời kiểm tra phiên bản PyTorch hiện có trong môi trường.

Việc sử dụng GPU giúp tăng tốc đáng kể quá trình huấn luyện mô hình Faster R-CNN do đây là mô hình có số lượng tham số lớn và yêu cầu khối lượng tính toán cao trong quá trình học.

Bước 2. Kết nối Google Drive

Để thuận tiện cho việc lưu trữ dữ liệu và mô hình, nhóm tiến hành kết nối Google Drive với môi trường Google Colab.

Sau khi kết nối thành công, toàn bộ dữ liệu huấn luyện, kết quả thực nghiệm và mô hình sau huấn luyện có thể được lưu trực tiếp lên Google Drive nhằm tránh mất dữ liệu khi phiên làm việc Colab kết thúc.

Bước 3. Cài đặt mô hình Faster R-CNN

Nhóm tiến hành cài đặt các thư viện cần thiết gồm Torchvision, PyCOCOTools và TorchMetrics để phục vụ cho quá trình xây dựng, huấn luyện và đánh giá mô hình Faster R-CNN.

Mô hình Faster R-CNN bao gồm bốn thành phần chính: Backbone ResNet-50 dùng để trích xuất đặc trưng ảnh, Feature Pyramid Network (FPN) hỗ trợ phát hiện đối tượng ở nhiều tỷ lệ khác nhau, Region Proposal Network (RPN) có nhiệm vụ sinh các vùng đề xuất chứa đối tượng và Fast R-CNN Head dùng để phân loại đối tượng cũng như tinh chỉnh vị trí Bounding Box.

Mô hình được khởi tạo với trọng số pretrained trên bộ dữ liệu COCO. Các trọng số này cung cấp khả năng trích xuất đặc trưng ban đầu và giúp quá trình huấn luyện hội tụ nhanh hơn.

Do bộ dữ liệu Safety Helmet Dataset chỉ bao gồm ba lớp đối tượng gồm Head, Helmet và Person, nhóm thay thế Classification Head mặc định của mô hình bằng một Classification Head mới phù hợp với bài toán hiện tại.

Trong Faster R-CNN, lớp Background được sử dụng nội bộ bởi mô hình để phân biệt giữa vùng chứa đối tượng và vùng không chứa đối tượng. Vì vậy số lượng lớp của mô hình được thiết lập là 4, bao gồm: Background, Head, Helmet, Person.

Sau khi hoàn tất cấu hình, mô hình được chuyển sang GPU để phục vụ quá trình huấn luyện.

Bước 4. Tải và chuẩn bị tập dữ liệu

Nhóm sử dụng bộ dữ liệu Safety Helmet Dataset ở định dạng COCO. Dataset được tải từ Google Drive và giải nén vào môi trường Google Colab.

Để đọc dữ liệu COCO, nhóm xây dựng lớp FasterRCNNDataset kế thừa từ CocoDetection của Torchvision. Lớp này có nhiệm vụ đọc ảnh và annotation từ bộ dữ liệu, đồng thời chuyển đổi annotation sang định dạng mà Faster R-CNN yêu cầu.

Trong phương thức **getitem()**, mỗi ảnh được chuyển sang Tensor thông qua hàm T.ToTensor(). Đồng thời, các annotation tương ứng được xử lý để tạo thành cấu trúc target gồm các trường boxes, labels, area, iscrowd và image_id theo đúng định dạng đầu vào của Faster R-CNN.

Trong quá trình xử lý dữ liệu, nhóm loại bỏ lớp person-head-helmet có category_id = 0. Đây là lớp trung gian xuất hiện trong cấu trúc dữ liệu COCO của bộ dữ liệu nhưng không thuộc ba lớp đối tượng cần phát hiện trong bài toán. Chỉ ba lớp Head, Helmet và Person được giữ lại để phục vụ quá trình huấn luyện.

Các Bounding Box trong COCO được chuyển đổi từ định dạng: (x, y, width, height) sang định dạng: (x1, y1, x2, y2) để phù hợp với yêu cầu đầu vào của Faster R-CNN.

Sau đó, nhóm xây dựng ba tập dữ liệu riêng biệt gồm train_dataset, val_dataset và test_dataset.

Để phục vụ quá trình huấn luyện theo mini-batch, nhóm xây dựng hàm collate_fn nhằm gom nhiều mẫu dữ liệu thành một batch. Hàm này giúp xử lý trường hợp mỗi ảnh có số lượng đối tượng khác nhau, vốn là đặc trưng thường gặp trong bài toán Object Detection.

Sau bước này, toàn bộ dữ liệu đã được chuyển đổi sang định dạng phù hợp với Faster R-CNN và sẵn sàng cho quá trình huấn luyện.

Bước 5. Huấn luyện mô hình Faster R-CNN

Sau khi hoàn thành việc xây dựng Dataset và DataLoader, nhóm tiến hành huấn luyện mô hình Faster R-CNN.

Trong quá trình cấu hình huấn luyện, mô hình được thiết lập với tối đa 100 epoch, batch size bằng 8 và tốc độ học (Learning Rate) bằng $1e-4$. Bộ tối ưu hóa AdamW được sử dụng để cập nhật trọng số mô hình trong suốt quá trình huấn luyện.

Để phục vụ huấn luyện, nhóm xây dựng `train_loader` bằng lớp `DataLoader` với `shuffle=True` nhằm xáo trộn dữ liệu ở mỗi epoch, giúp mô hình học ổn định hơn và giảm hiện tượng phụ thuộc vào thứ tự dữ liệu.

Khác với YOLOv8 và DETR sử dụng các pipeline huấn luyện được đóng gói sẵn trong thư viện Ultralytics và Hugging Face Transformers, Faster R-CNN trong Torchvision yêu cầu người dùng tự xây dựng vòng lặp huấn luyện bằng PyTorch.

Trong mỗi epoch, dữ liệu từ `train_loader` được đưa vào mô hình theo từng mini-batch. Các ảnh và annotation tương ứng được chuyển sang GPU trước khi thực hiện huấn luyện.

Đối với mỗi batch dữ liệu, mô hình thực hiện lan truyền xuôi (Forward Pass) thông qua lệnh: `loss_dict = model(images, targets)`

Kết quả trả về là một Loss Dictionary bao gồm nhiều thành phần hàm mất mát khác nhau như Classification Loss, Box Regression Loss, Objectness Loss và Region Proposal Loss.

Tổng hàm mất mát được tính bằng cách cộng tất cả các thành phần Loss: `loss = sum(loss_dict.values())`. Trong đó, Classification Loss đánh giá khả năng phân loại đối tượng, Box Regression Loss đánh giá độ chính xác của Bounding Box, Objectness Loss đánh giá khả năng phân biệt giữa vùng chứa đối tượng và vùng nền, còn Region Proposal Loss đánh giá chất lượng các vùng đề xuất được sinh bởi Region Proposal Network (RPN).

Tiếp theo, quá trình lan truyền ngược (Backpropagation) được thực hiện nhằm tính toán gradient của các tham số. Bộ tối ưu hóa AdamW sử dụng các gradient này để cập nhật trọng số mô hình thông qua các bước `zero_grad()`, `backward()` và `step()`.

Sau mỗi epoch, giá trị Loss trung bình trên toàn bộ tập Train được tính toán và lưu lại trong danh sách `train_losses`. Các giá trị này sẽ được sử dụng để xây dựng biểu đồ Learning Curve trong bước đánh giá.

Khi quá trình huấn luyện kết thúc, trọng số của mô hình được lưu vào tệp `helmet_faster_rcnn.pth` để phục vụ cho các bước đánh giá và dự đoán tiếp theo.

Bước 6. Đánh giá kết quả huấn luyện

- Đánh giá mô hình trên tập Validation

Sau khi hoàn thành quá trình huấn luyện, nhóm tiến hành đánh giá mô hình trên tập Validation.

Để đảm bảo tính nhất quán với các mô hình YOLOv8 và DETR, nhóm sử dụng các chỉ số $mAR@100$, $mAP@0.5$ và $mAP@0.5:0.95$. Các chỉ số này được tính toán bằng thư viện TorchMetrics thông qua lớp MeanAveragePrecision.

Trong quá trình đánh giá, các ảnh trong tập Validation được đưa qua mô hình để tạo ra các dự đoán. Các Bounding Box dự đoán, độ tin cậy (scores) và nhãn lớp (labels) được trích xuất từ đầu ra của mô hình và truyền vào đối tượng metric thông qua hàm `metric.update()`.

Sau khi toàn bộ tập Validation được xử lý, các chỉ số đánh giá cuối cùng được tính toán bằng lệnh `metric.compute()`. Từ kết quả này, nhóm thu được các chỉ số $mAR@100$, $mAP@0.5$ và $mAP@0.5:0.95$ phản ánh chất lượng phát hiện đối tượng của mô hình.

- Learning Curve

Để đánh giá khả năng hội tụ của mô hình, nhóm tiến hành trực quan hóa lịch sử huấn luyện thông qua các giá trị Training Loss được lưu lại sau mỗi epoch trong danh sách `train_losses`.

Các giá trị này được sử dụng để xây dựng biểu đồ Learning Curve bằng thư viện Matplotlib. Biểu đồ cho phép quan sát xu hướng giảm của hàm mất mát theo thời gian huấn luyện, từ đó đánh giá khả năng học và mức độ hội tụ của mô hình.

Bước 7. Đánh giá mô hình trên tập Test

Sau khi hoàn tất quá trình đánh giá trên tập Validation, nhóm tiếp tục sử dụng mô hình đã huấn luyện để đánh giá trên tập Test.

Các chỉ số $mAR@100$, $mAP@0.5$ và $mAP@0.5:0.95$ tiếp tục được sử dụng nhằm phản ánh khả năng tổng quát hóa của mô hình đối với dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.

Kết quả trên tập Test được sử dụng làm cơ sở chính để so sánh hiệu năng giữa Faster R-CNN, YOLOv8 và DETR trong phần thực nghiệm.

Bước 8. Dự đoán trên ảnh thử nghiệm

Sau khi đánh giá trên tập Test, nhóm tiến hành dự đoán trên một ảnh được lựa chọn ngẫu nhiên từ tập Test.

Mô hình thực hiện dự đoán vị trí Bounding Box, nhãn lớp và độ tin cậy của từng đối tượng xuất hiện trong ảnh. Kết quả dự đoán được trực quan hóa trực tiếp trên ảnh với màu sắc khác nhau cho từng lớp đối tượng nhằm hỗ trợ việc quan sát và phân tích kết quả nhận dạng.

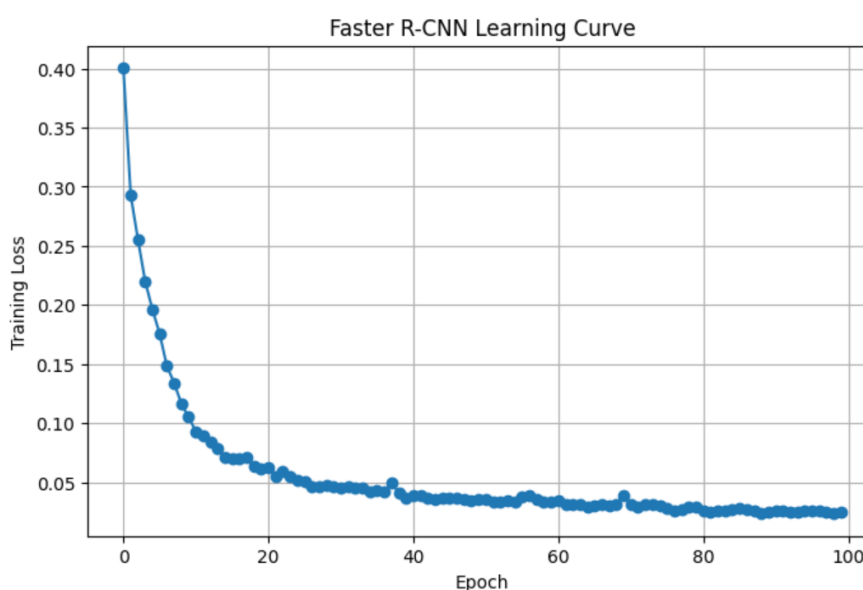
Bước 9. Lưu mô hình vào Google Drive

Sau khi hoàn tất quá trình huấn luyện và đánh giá, nhóm tiến hành sao chép tệp trọng số `helmet_faster_rcnn.pth` vào Google Drive.

Việc lưu trữ mô hình trên Google Drive giúp thuận tiện cho quá trình sử dụng lại mô hình, thực hiện suy luận hoặc tiếp tục nghiên cứu trong các giai đoạn tiếp theo.

3.5.2. Kết quả đánh giá

a) Learning Curve



Hình 9. Sự thay đổi Training Loss trong quá trình huấn luyện Faster R-CNN

Quá trình huấn luyện của Faster R-CNN diễn ra ổn định với giá trị Training Loss giảm dần theo số epoch. Trong giai đoạn đầu, loss giảm nhanh cho thấy mô hình nhanh chóng học được các đặc trưng quan trọng của dữ liệu. Ở các epoch sau, tốc độ giảm loss chậm lại và dần hội tụ, phản ánh quá trình tối ưu hóa đã đạt trạng thái ổn định. Kết quả này cho thấy mô hình có khả năng học hiệu quả trên bộ dữ liệu Safety Helmet và không xuất hiện dấu hiệu mất ổn định trong quá trình huấn luyện.

b) Đánh giá mô hình trên tập Validation

Bảng 7. Kết quả đánh giá mô hình Faster R-CNN trên tập Validation

Chỉ số	Giá trị
mAR@100	0.4589
mAP@50	0.7595
mAP@50:95	0.3867

Kết quả đánh giá trên tập Validation cho thấy mô hình Faster R-CNN đạt mAP@50 bằng 75.95%, mAP@50:95 bằng 38.67% và mAR@100 bằng 45.89%. Giá trị mAP@50 tương đối cao cho thấy mô hình có khả năng phát hiện và phân loại chính xác phần lớn các đối tượng trong bộ dữ liệu. Tuy nhiên, khi áp dụng các ngưỡng IoU nghiêm ngặt hơn, giá trị mAP@50:95 giảm xuống còn 38.67%, phản ánh chất lượng định vị Bounding Box vẫn còn hạn chế đối với một số đối tượng có kích thước nhỏ hoặc bị che khuất. Nhìn chung, mô hình đã học được các đặc trưng cần thiết của dữ liệu và đạt hiệu năng đủ tốt để tiếp tục đánh giá trên tập Test.

c) Đánh giá mô hình trên tập Test

Bảng 8. Kết quả đánh giá mô hình Faster R-CNN trên tập Test

Chỉ số	Giá trị
mAR@100	0.4938
mAP@50	0.8059
mAP@50:95	0.4200

Trên tập Test, Faster R-CNN đạt mAP@50 bằng 80.59%, mAP@50:95 bằng 42.00% và mAR@100 bằng 49.38%. Kết quả cho thấy mô hình duy trì được khả năng phát hiện đối tượng tốt trên dữ liệu chưa xuất hiện trong quá trình huấn luyện. Giá trị mAP@50 vượt mức 80% chứng tỏ phần lớn các đối tượng Helmet, Head và Person được phát hiện chính xác. Mặc dù giá trị mAR@100 chưa cao, cho thấy mô hình vẫn còn bỏ sót một số đối tượng, Faster R-CNN vẫn thể hiện khả năng tổng quát hóa ổn định và đạt hiệu quả phát hiện đối tượng tương đối tốt trên bộ dữ liệu Safety Helmet.

3.6. So sánh các mô hình YOLOv8, DETR và Faster R-CNN

3.6.1. So sánh đặc điểm của các mô hình

Trong lĩnh vực phát hiện đối tượng (Object Detection), các mô hình thường được đánh giá dựa trên các chỉ số như Precision, Recall, mAP, tốc độ suy luận và khả năng triển khai thực tế. Ba hướng tiếp cận tiêu biểu hiện nay gồm mô hình hai giai đoạn (Two-Stage Detector), mô hình một giai đoạn (One-Stage Detector) và mô hình dựa trên Transformer.

Faster R-CNN được đề xuất năm 2015, thuộc nhóm mô hình hai giai đoạn, sử dụng Region Proposal Network (RPN) để tạo vùng ứng viên trước khi thực hiện phân loại và định vị đối tượng [4]. Năm 2020, DETR ra đời, áp dụng kiến trúc Transformer và cơ chế Hungarian Matching, cho phép thực hiện phát hiện đối tượng theo hướng End-to-End mà không cần Anchor Box hay Non-Maximum Suppression (NMS) [3]. Đến năm 2023, YOLOv8 được giới thiệu với kiến trúc Anchor-Free, tập trung tối ưu tốc độ suy luận đồng thời duy trì độ chính xác cao [2].

Sự xuất hiện lần lượt của Faster R-CNN, DETR và YOLOv8 phản ánh quá trình phát triển của các mô hình phát hiện đối tượng từ kiến trúc CNN truyền thống sang các phương pháp End-to-End và Transformer hiện đại.

Ưu điểm và hạn chế của từng mô hình

Faster R-CNN

Ưu điểm lớn nhất của Faster R-CNN là độ chính xác cao và khả năng phát hiện các vật thể nhỏ tương đối tốt nhờ cơ chế Region Proposal Network. Mô hình thường đạt kết quả ổn định trên các tập dữ liệu có số lượng đối tượng lớn và kích thước đối tượng đa dạng [4].

Tuy nhiên, Faster R-CNN có tốc độ suy luận thấp hơn các mô hình một giai đoạn do phải thực hiện quá trình sinh vùng đề xuất trước khi phân loại. Điều này khiến mô hình khó đáp ứng các ứng dụng thời gian thực như giám sát video hoặc xe tự hành.

DETR

DETR sở hữu kiến trúc đơn giản hơn so với các phương pháp truyền thống khi loại bỏ hoàn toàn Anchor Box và NMS. Transformer cho phép mô hình học được mối quan hệ toàn cục giữa các đối tượng trong ảnh thay vì chỉ dựa trên vùng lân cận như CNN. Ngoài ra, DETR có khả năng mở rộng tốt sang các tác vụ khác như Instance Segmentation hoặc Panoptic Segmentation [3].

Tuy nhiên, nhược điểm lớn của DETR là thời gian huấn luyện dài và tốc độ hội tụ chậm. Nghiên cứu gốc cho thấy DETR cần số lượng epoch lớn hơn đáng kể so với các

detector CNN truyền thống. Bên cạnh đó, mô hình gặp khó khăn khi phát hiện các vật thể nhỏ do đặc trưng không gian bị suy giảm trong quá trình Transformer xử lý ảnh [5].

YOLOv8

YOLOv8 được thiết kế hướng đến cân bằng giữa tốc độ và độ chính xác. Mô hình sử dụng cơ chế Anchor-Free giúp giảm độ phức tạp trong huấn luyện và suy luận. Bên cạnh đó, YOLOv8 hỗ trợ phát hiện đa tỉ lệ hiệu quả, cho phép nhận diện tốt cả các đối tượng lớn và nhỏ trong ảnh. Tốc độ xử lý nhanh giúp YOLOv8 phù hợp với các ứng dụng thời gian thực [2].

Hạn chế của YOLOv8 là khả năng mô hình hóa ngữ cảnh toàn cục chưa mạnh bằng các mô hình Transformer như DETR. Trong một số bài toán phức tạp yêu cầu hiểu quan hệ giữa nhiều đối tượng trong ảnh, Transformer có thể mang lại lợi thế nhất định.

Bảng 9. Bảng so sánh tổng quan giữa các mô hình

Tiêu chí	Faster R-CNN	DETR	YOLOv8
Năm ra đời	2015	2020	2023
Kiến trúc	CNN hai giai đoạn	Transformer End-to-End	CNN một giai đoạn
Region Proposal	Có (RPN)	Không	Không
Anchor Box	Có	Không	Không
NMS	Có	Không	Có
Tốc độ suy luận	Trung bình	Trung bình	Rất nhanh
Thời gian huấn luyện	Trung bình	Dài	Ngắn
Phát hiện vật thể nhỏ	Tốt	Hạn chế	Khá tốt
Quan hệ ngữ cảnh toàn cục	Hạn chế	Rất tốt	Trung bình
Triển khai thời gian thực	Khó	Khó	Tốt

Từ những đặc điểm trên có thể thấy Faster R-CNN phù hợp khi ưu tiên độ chính xác và khả năng phát hiện vật thể nhỏ. DETR mang lại cách tiếp cận hiện đại dựa trên Transformer với kiến trúc End-to-End đơn giản nhưng đòi hỏi thời gian huấn luyện dài hơn. Trong khi đó, YOLOv8 đạt được sự cân bằng giữa tốc độ và độ chính xác, là lựa chọn phổ biến cho các ứng dụng thực tế yêu cầu xử lý thời gian thực.

3.6.2. So sánh kết quả thực nghiệm trên bộ dữ liệu Safety Helmet

Bảng 10. So sánh kết quả đánh giá trên tập Test

Mô hình	Recall	mAR@100	mAP@50	mAP@50:95
YOLOv8	0.846	-	0.887	0.467
Faster R-CNN	-	0.4938	0.8059	0.4200
DETR	-	0.3567	0.5317	0.2681

Kết quả thực nghiệm cho thấy YOLOv8 đạt hiệu năng tốt nhất trên bộ dữ liệu Safety Helmet. Mô hình đạt $mAP@50 = 88.7\%$ và $mAP@50:95 = 46.7\%$, cao hơn cả Faster R-CNN và DETR. Đồng thời, Precision đạt 83.6% và Recall đạt 84.6% , cho thấy YOLOv8 không chỉ phát hiện chính xác mà còn nhận diện được phần lớn các đối tượng xuất hiện trong ảnh. Điều này phản ánh khả năng cân bằng tốt giữa độ chính xác phát hiện và khả năng thu hồi đối tượng của mô hình. Bên cạnh đó, YOLOv8 cũng cho thấy khả năng hội tụ nhanh và ổn định trong quá trình huấn luyện, phù hợp với các bài toán phát hiện đối tượng trong thực tế.

Faster R-CNN đạt $mAP@50 = 80.59\%$ và $mAP@50:95 = 42.00\%$, đứng thứ hai trong ba mô hình được đánh giá. Kết quả này cho thấy kiến trúc Two-stage Detector vẫn duy trì khả năng phát hiện đối tượng hiệu quả nhờ cơ chế Region Proposal Network (RPN), giúp xác định các vùng chứa đối tượng trước khi thực hiện phân loại và hồi quy Bounding Box. Tuy nhiên, hiệu năng của Faster R-CNN vẫn thấp hơn YOLOv8 trên cả hai chỉ số mAP. Ngoài ra, giá trị mAR@100 đạt 49.38% cho thấy khả năng thu hồi đối tượng của mô hình vẫn còn hạn chế so với YOLOv8. Mặc dù vậy, Faster R-CNN vẫn đạt kết quả tương đối tốt và thể hiện tính ổn định trên bộ dữ liệu Safety Helmet.

DETR cho kết quả thấp nhất với $mAP@50 = 53.17\%$ và $mAP@50:95 = 26.81\%$. Mặc dù mô hình có ưu điểm về thiết kế end-to-end và không phụ thuộc vào Anchor Box hay Non-Maximum Suppression (NMS), kết quả thực nghiệm cho thấy DETR gặp nhiều khó khăn hơn trong việc phát hiện các đối tượng trên bộ dữ liệu Safety Helmet. Đặc biệt, các đối

tượng có kích thước nhỏ như lớp Head thường khó được phát hiện chính xác. Điều này cũng được phản ánh qua giá trị $mAR@100$ chỉ đạt 35.67%, thấp hơn đáng kể so với YOLOv8 và Faster R-CNN. Kết quả này phù hợp với nhận định trong các nghiên cứu trước đây rằng DETR nguyên bản thường yêu cầu thời gian huấn luyện dài hơn và gặp hạn chế khi xử lý các đối tượng có kích thước nhỏ.

Nhìn chung, YOLOv8 là mô hình đạt hiệu quả cao nhất trên bộ dữ liệu Safety Helmet nhờ khả năng phát hiện chính xác, tốc độ hội tụ nhanh và hiệu năng ổn định trên cả tập Validation và Test. Faster R-CNN đạt kết quả khá tốt, cho thấy ưu thế của kiến trúc Two-stage Detector trong việc phát hiện đối tượng, tuy nhiên vẫn chưa đạt được hiệu năng tổng thể như YOLOv8. Trong khi đó, DETR thể hiện tiềm năng của hướng tiếp cận Transformer-based Detector nhưng chưa thực sự phù hợp với bộ dữ liệu có quy mô nhỏ và chứa nhiều đối tượng kích thước nhỏ như Safety Helmet. Từ các kết quả thu được, YOLOv8 được đánh giá là mô hình phù hợp nhất cho bài toán phát hiện mũ bảo hiểm trong phạm vi bộ dữ liệu được sử dụng trong bài thực hành.

Chương 4. KẾT LUẬN

Trong bài thực hành này, nhóm đã nghiên cứu và triển khai bài toán phát hiện đối tượng (Object Detection) trên bộ dữ liệu **Safety Helmet Dataset** được thu thập từ Roboflow Universe. Bộ dữ liệu gồm 1.088 ảnh được chia thành ba tập Train, Validation và Test, với ba lớp đối tượng cần phát hiện là Helmet, Head và Person. Dữ liệu được sử dụng dưới hai định dạng khác nhau gồm YOLO và COCO nhằm phù hợp với yêu cầu đầu vào của các mô hình YOLOv8, DETR và Faster R-CNN.

Trên cơ sở bộ dữ liệu này, nhóm đã tiến hành huấn luyện, đánh giá và so sánh ba mô hình phát hiện đối tượng tiêu biểu thuộc ba hướng tiếp cận khác nhau. Trong đó, **YOLOv8** đại diện cho nhóm One-stage Detector, **Faster R-CNN** đại diện cho nhóm Two-stage Detector và **DETR** đại diện cho nhóm Transformer-based Detector. Các mô hình được đánh giá dựa trên các chỉ số $mAP@0.5$, $mAP@0.5:0.95$, Precision, Recall hoặc $mAR@100$ nhằm phản ánh khả năng phát hiện và định vị đối tượng.

Kết quả thực nghiệm cho thấy YOLOv8 đạt hiệu năng tốt nhất trên bộ dữ liệu Safety Helmet với $mAP@0.5$ đạt 88.7% và $mAP@0.5:0.95$ đạt 46.7% trên tập Test. Faster R-CNN đạt kết quả khá tốt với $mAP@0.5$ bằng 80.59% và $mAP@0.5:0.95$ bằng 42.00%, cho thấy khả năng phát hiện đối tượng ổn định của kiến trúc Two-stage Detector. Trong khi đó, DETR đạt $mAP@0.5$ bằng 53.17% và $mAP@0.5:0.95$ bằng 26.81%, thấp hơn hai mô hình còn lại và gặp nhiều khó khăn khi xử lý các đối tượng có kích thước nhỏ như lớp Head.

Từ kết quả so sánh, có thể nhận thấy YOLOv8 là mô hình phù hợp nhất đối với bộ dữ liệu Safety Helmet nhờ khả năng hội tụ nhanh, hiệu năng phát hiện cao và quy trình huấn luyện đơn giản. Faster R-CNN vẫn duy trì độ chính xác tốt nhưng yêu cầu thời gian huấn luyện và suy luận lớn hơn. Đối với DETR, mặc dù sở hữu kiến trúc hiện đại dựa trên Transformer và cơ chế phát hiện end-to-end, mô hình vẫn cần nhiều dữ liệu hoặc các phiên bản cải tiến để đạt hiệu quả cao hơn trên các bài toán có nhiều đối tượng nhỏ.

Thông qua bài thực hành, nhóm đã nắm được quy trình xây dựng dữ liệu, huấn luyện, đánh giá và so sánh các mô hình Object Detection hiện đại. Đồng thời, nhóm hiểu rõ hơn về ưu điểm, hạn chế và đặc điểm của từng hướng tiếp cận trong bài toán phát hiện đối tượng, từ đó tạo nền tảng cho việc nghiên cứu và triển khai các hệ thống thị giác máy tính trong các ứng dụng thực tế.

TÀI LIỆU THAM KHẢO

- [1] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object Detection in 20 Years: A Survey," *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023.
- [2] [Ultralytics YOLO Documentation](#), Ultralytics Inc., 2024. Accessed: Jun. 2026.
- [3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020, pp. 213–229.
- [4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 39, no. 6, pp. 1137–1149, 2017. (Conference version published in NIPS 2015).
- [5] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable Transformers for End-to-End Object Detection," *International Conference on Learning Representations (ICLR)*, 2021.